

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: KỸ THUẬT PHẦN MỀM**

**ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG CHATBOT THÔNG
MINH**

Nhóm 2:

Trần Lê Anh	22010083
Luyện Thanh Thùy	22010272

Giảng viên hướng dẫn: TS. Trịnh Thanh Bình

Hà Nội, ngày 11 tháng 6 năm 2025

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và Tên	MSSV	Nhiệm vụ	Chức vụ
Trần Lê Anh	22010083	- Nghiên cứu cơ sở lý thuyết. - Xây dựng Backend (API, xử lý file, Conversation, Google Search API). - Thiết kế và xây dựng Frontend (Services, Libs, 1 phần Component). - Viết nội dung báo cáo (Phần 1, 2, 4, 6).	Leader
Luyện Thanh Thùy	22010272	- Xây dựng Backend (API, RAG, LLM, Database). - Thiết kế giao diện người dùng (UI/UX). - Thiết kế và xây dựng Frontend (Types, 1 phần Component). - Viết nội dung báo cáo (Phần 3, 5, 7, 8) và tổng hợp lại báo cáo.	Member

Contents

Danh mục Hình ảnh	7
Danh mục Bảng biểu	8
1 TỔNG QUAN VỀ ĐỀ TÀI	9
1.1 Bối cảnh và Tính cấp thiết của đề tài	9
1.2 Mục tiêu của đề tài	10
1.3 Phạm vi và Giới hạn của đề tài	10
1.3.1 Phạm vi	10
1.3.2 Giới hạn	11
2 CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN CÔNG NGHỆ	12
2.1 Cơ sở Lý thuyết	12
2.1.1 Mô hình Ngôn ngữ Lớn (Large Language Models - LLM) .	12
2.1.1.1 Kiến trúc Transformer: Cuộc cách mạng về xử lý chuỗi	12
2.1.1.2 Các Mô hình Tiền huấn luyện: BERT và GPT .	14
2.1.1.3 Luật Quy mô và các Khả năng Bất ngờ (Emergent Abilities)	15
2.1.1.4 Căn chỉnh với Ý định của Con người: RLHF . .	15
2.1.1.5 Các Thách thức và Giới hạn Cố hữu của LLM .	16
2.1.2 Retrieval-Augmented Generation (RAG)	17
2.1.2.1 Giai đoạn 1: Xây dựng và Lập chỉ mục Cơ sở tri thức (Indexing Phase)	18
2.1.2.2 Giai đoạn 2: Truy xuất và Tạo sinh (Retrieval and Generation Phase)	19

2.1.2.3	Ưu điểm của Kiến trúc RAG	19
2.1.3	Vector Embeddings và Cơ sở dữ liệu Vector	20
2.1.3.1	Vector Embeddings: Mã hóa Ngữ nghĩa thành Dạng Toán học	20
2.1.3.2	Cơ sở dữ liệu Vector: Lưu trữ và Truy xuất Hiệu quả	20
2.2	Ngăn xếp Công nghệ (Technology Stack)	21
2.2.1	Phát triển Backend	22
2.2.1.1	Mô hình Embedding: Sentence-BERT	22
2.2.1.2	Tích hợp Google Custom Search API để mở rộng tri thức	23
2.2.1.3	Tích hợp vào Pipeline RAG	23
2.2.2	Phát triển Frontend	24
2.2.3	Đóng gói và Triển khai	24
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	26
3.1	Phân tích Yêu cầu	26
3.1.1	Yêu cầu Chức năng (Functional Requirements)	26
3.1.2	Yêu cầu Phi chức năng (Non-functional Requirements) . .	27
3.2	Phân tích và Thiết kế hệ thống	28
3.2.1	Sơ đồ hoạt động	28
3.2.2	Thiết kế Usecase	29
3.2.2.1	Mô tả Use Case chi tiết	29
3.2.2.2	Usecase tổng quát	29
3.2.2.3	Usecase: Trò chuyện với MinBot	30
3.2.2.4	Usecase: RAG	30
3.2.2.5	UseCase: Tìm kiếm thông tin Web	31
3.2.2.6	Usecase: Xem lại lịch sử hội thoại	31
3.3	Thiết kế Luồng dữ liệu (Data Flow Diagram - DFD)	32
3.3.1	Mô tả DFD – Sơ đồ luồng dữ liệu tổng quát	32
3.3.2	Các thực thể chính trong DFD	33

4	Kiến trúc Hệ thống	34
4.1	Kiến trúc tổng thể	34
4.2	Kiến trúc Backend	35
4.2.1	Cấu trúc phân lớp	35
4.2.2	Thiết kế Cơ sở dữ liệu	35
4.2.3	Các thành phần dịch vụ	36
4.2.4	Các luồng xử lý chính	37
4.2.4.1	Pipeline xử lý tệp tin	37
4.2.4.2	Pipeline RAG	37
4.3	Kiến trúc Frontend	38
4.3.1	Cấu trúc Component	38
4.3.2	Quản lý trạng thái và Tương tác API	39
4.4	Triển khai và Vận hành	39
4.4.1	Containerization với Docker	39
4.4.2	Logging và Giám sát	40
5	THỰC THI HỆ THỐNG	41
5.1	Kiến trúc tổng thể	41
5.2	Pipeline xử lý lỗi: Retrieval-Augmented Generation	42
5.2.1	Luồng xử lý truy vấn RAG	42
5.2.2	Luồng xử lý và Vector hóa tài liệu	43
5.3	Môi trường triển khai (Docker Compose)	44
5.4	Bảo mật và Giám sát	44
5.4.1	Bảo mật	44
5.4.2	Giám sát	44
5.5	Tối ưu hóa và Xử lý Bất đồng bộ	45
6	KIỂM THỬ	46
6.1	Kế hoạch Kiểm thử	46
6.1.1	Mục tiêu kiểm thử	46
6.1.2	Phạm vi và Phương pháp kiểm thử	46

6.2	Thiết kế Ca kiểm thử (Test Cases)	47
6.2.1	Chức năng: Tải và Xử lý Tài liệu	47
6.2.2	Chức năng: Trò chuyện với Chatbot (Lỗi RAG)	48
6.2.3	Chức năng: Quản lý Hội thoại	48
6.2.4	Giao diện: Tải và Xử lý Tài liệu (Frontend)	48
6.2.5	Giao diện: Trò chuyện với Chatbot	49
6.2.6	Giao diện: Quản lý Hội thoại	49
6.3	Kết quả và Phân tích	49
6.3.1	Bảng tổng hợp kết quả kiểm thử	49
6.3.2	Phân tích lỗi và Đề xuất cải tiến	50
6.4	Tổng kết và Hướng phát triển tương lai	52
6.4.1	Tổng kết	52
6.4.2	Hướng phát triển tương lai	52
7	TRIỂN KHAI VÀ CÀI ĐẶT	54
7.1	Tổng quan	54
7.2	Môi trường Phát triển Cục bộ (Local Development)	55
7.2.1	Công cụ yêu cầu	55
7.2.2	Hướng dẫn cài đặt	55
7.3	Triển khai với Docker	56
7.3.1	Phân tích Dockerfile	56
7.3.2	Phân tích docker-compose.yml	56
7.3.3	Hướng dẫn triển khai bằng Docker	57
8	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
8.1	Kết quả Đạt được	61
8.2	Hạn chế của dự án	62
8.3	Lộ trình Phát triển trong Tương lai	63
8.3.1	Giai đoạn 1: Hoàn thiện Sản phẩm Cốt lõi	63
8.3.2	Giai đoạn 2: Nâng cấp và Mở rộng	63
	Tài liệu tham khảo	65

List of Figures

2.1	Kiến trúc Transformer	13
2.2	(Trái) Scaled Dot-Product Attention. (Phải) Multi-Head Attention.	14
2.3	Hiện tượng ảo giác của model o3 và o4-mini	16
2.4	RAG Pipeline	18
3.1	Sơ đồ Activity diagram mô tả tương tác người dùng với hệ thống	28
3.2	Sơ đồ Usecase tổng quát	29
3.3	Sơ đồ Usecase trò chuyện với MinBot	30
3.4	Sơ đồ Usecase Truy xuất dữ liệu (RAG)	30
3.5	Sơ đồ Usecase Tìm kiếm thông tin từ Web	31
3.6	Sơ đồ Usecase xem lịch sử hội thoại	31
3.7	DFD chính của MinBot	32
4.1	Kiến trúc tổng thể của hệ thống MinBot.	34
4.2	Các lớp kiến trúc của Backend.	35
4.3	Sơ đồ tương tác giữa các thành phần trong lớp Service của Backend.	36
4.4	Pipeline xử lý tệp tin đầu vào.	37
4.5	Pipeline sinh truy vấn tăng cường (RAG) trong MinBot.	38
4.6	Sơ đồ phân cấp các thành phần chính của Frontend.	39
5.1	Kiến trúc triển khai tổng thể của MinBot	41
5.2	Pipeline RAG song song truy xuất tri thức nội bộ và web	43
7.1	Sơ đồ kiến trúc triển khai hệ thống bằng Docker Compose.	54
7.2	Giao diện chính của ứng dụng MinBot sau khi triển khai thành công.	57

7.3	Giao diện chat của MinBot.	58
7.4	Giao diện conversation của MinBot.	58
7.5	Giao diện tài liệu được gắn vào message của MinBot.	59
7.6	Giao diện upload tài liệu của MinBot.	59
7.7	Giao diện cài đặt của MinBot.	59
7.8	Giao diện tài liệu API (Swagger UI) của backend tại địa chỉ /docs.	60

List of Tables

3.1	Yêu cầu Chức năng (Functional Requirements)	26
3.2	Yêu cầu Phi chức năng (Non-functional Requirements)	27
3.4	Chi tiết các Use Case chính của hệ thống	29
3.5	Bảng mô tả vai trò của các thực thể trong hệ thống	33
4.1	Schema bảng conversations trong SQLite.	36
4.2	Schema bảng messages trong SQLite.	36
6.1	Test cases cho chức năng Tải và Xử lý Tài liệu	47
6.2	Test cases cho chức năng Trò chuyện	48
6.3	Test cases cho chức năng Quản lý Hội thoại	48
6.4	Test cases giao diện Tải và Xử lý Tài liệu	49
6.5	Test cases giao diện Trò chuyện	49
6.6	Test cases giao diện Quản lý Hội thoại	49
6.7	Kết quả thực thi các ca kiểm thử	50

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Bối cảnh và Tính cấp thiết của đề tài

Sự phát triển vượt bậc của các Mô hình Ngôn ngữ Lớn ([LargeLanguageModel s-LLM](#)) đã mở ra một kỷ nguyên mới cho các ứng dụng xử lý ngôn ngữ tự nhiên, điển hình là các hệ thống Chatbot và trợ lý ảo ngày càng thông minh. Tuy nhiên, các mô hình này vẫn tồn tại những thách thức cố hữu, làm hạn chế độ tin cậy và khả năng ứng dụng trong các tác vụ đòi hỏi sự chính xác cao:

- **Hiện tượng "ảo giác":** ([Hallucination](#)) Các mô hình có xu hướng tự sinh ra thông tin sai lệch, không dựa trên dữ liệu thực tế, gây mất niềm tin cho người dùng và tiềm ẩn rủi ro khi ứng dụng trong các lĩnh vực quan trọng.
- **Giới hạn kiến thức** ([KnowledgeCutoff](#)): Do dữ liệu huấn luyện bị "đóng băng" tại một thời điểm nhất định, [LLM](#) không thể truy cập các thông tin mới hoặc các nguồn tri thức mang tính nội bộ, độc quyền của một tổ chức hay cá nhân.

Trong thực tế, nhu cầu về một hệ thống hỏi-đáp có khả năng khai thác các nguồn kiến thức chuyên biệt (như tài liệu học tập, quy trình nội bộ, văn bản pháp quy) và cập nhật thông tin thời gian thực từ Internet là vô cùng lớn. Các giải pháp truyền thống như huấn luyện lại ([fine-tuning](#)) toàn bộ mô hình thường đòi hỏi chi phí tính toán khổng lồ, dữ liệu lớn và quy trình kỹ thuật phức tạp, không phù hợp với nhiều bài toán ứng dụng.

Để giải quyết bài toán này, kỹ thuật Truy xuất-Tăng cường-Tạo sinh (Retrieval-Augmented Generation - [RAG](#)) đã nổi lên như một giải pháp đột phá và hiệu quả. Thay vì chỉ dựa vào kiến thức nội tại, [RAG](#) cho phép [LLM](#) truy xuất các thông tin liên quan từ một cơ sở tri thức bên ngoài trước khi tạo ra câu trả lời. Cách tiếp cận này giúp "neo" câu trả lời của mô hình vào các bằng chứng xác thực, qua đó giảm thiểu đáng kể hiện tượng ảo giác và cho phép hệ thống cập nhật kiến thức một cách linh hoạt mà không cần huấn luyện lại.

Xuất phát từ những phân tích trên, đề tài "Xây dựng Ứng dụng Chatbot thông minh tích hợp RAG, Tìm kiếm Web và Phân tích Tài liệu" được thực hiện. Đề tài hướng tới việc xây dựng một công cụ hỏi-đáp hoàn chỉnh, chính xác và có khả năng tùy biến cao, phục vụ các đối tượng đa dạng như sinh viên tra cứu tài liệu, nhân viên khai thác tri thức doanh nghiệp, hoặc bất kỳ ai cần một trợ lý ảo chuyên sâu.

1.2 Mục tiêu của đề tài

Đề tài đặt ra mục tiêu tổng quát và các mục tiêu cụ thể như sau:

- **Mục tiêu tổng quát:** Phát triển và kiểm chứng một hệ thống Chatbot hoàn chỉnh, ứng dụng thành công kiến trúc [RAG](#), có khả năng cung cấp câu trả lời chính xác dựa trên nguồn tài liệu do người dùng cung cấp hoặc kết quả tìm kiếm từ Internet.
- **Các mục tiêu cụ thể:**
 1. **Xây dựng Backend API:** Triển khai hệ thống API bằng [Python](#) và [FastAPI](#) để xử lý các logic nghiệp vụ cốt lõi, bao gồm: tiếp nhận và tiền xử lý tài liệu ([PDF](#), [DOCX](#), [TXT](#)), điều phối luồng xử lý [RAG](#), tích hợp [LLM](#), và quản lý lịch sử hội thoại.
 2. **Phát triển Frontend:** Xây dựng giao diện người dùng (UI) trực quan và thân thiện bằng [React](#) và [TypeScript](#), cho phép người dùng thực hiện các tác vụ: tải lên và quản lý tài liệu, tương tác với Chatbot, và xem lại lịch sử trò chuyện.
 3. **Triển khai pipeline [RAG](#):** Xây dựng luồng xử lý [RAG](#) hoàn chỉnh, từ việc vector hóa văn bản, lưu trữ và truy vấn hiệu quả trên cơ sở dữ liệu vector (sử dụng thư viện [FAISS](#)), đến việc tích hợp kết quả truy xuất vào ngữ cảnh của [LLM](#).
 4. **Tích hợp tìm kiếm Web:** Mở rộng khả năng của Chatbot bằng cách tích hợp chức năng tìm kiếm trực tuyến (ví dụ: qua [GoogleSearchAPI](#)) để trả lời các câu hỏi nằm ngoài phạm vi tài liệu được cung cấp.
 5. **Đóng gói và triển khai:** Sử dụng [Docker](#) và [DockerCompose](#) để container hóa toàn bộ ứng dụng (backend, frontend, và các dịch vụ phụ thuộc), đảm bảo tính di động và đơn giản hóa quá trình cài đặt, triển khai.

1.3 Phạm vi và Giới hạn của đề tài

1.3.1 Phạm vi

- **Chức năng cốt lõi:** Hệ thống tập trung vào các luồng chức năng chính:
 - (1) Tải lên, xử lý và vector hóa tài liệu ([.pdf](#), [.docx](#), [.txt](#));
 - (2) Hỏi-đáp dựa trên kho tài liệu đã xây dựng;
 - (3) Tùy chọn hỏi-đáp dựa trên kết quả tìm kiếm web;
 - (4) Lưu trữ và hiển thị lịch sử hội thoại.
- **Ngăn xếp công nghệ:** Dự án giới hạn trong việc sử dụng các công nghệ đã xác định: [Python](#), [FastAPI](#), [React](#), [TypeScript](#), [FAISS](#), và [Docker](#).

1.3.2 Giới hạn

- **Giao diện người dùng:** Giao diện được thiết kế để đảm bảo tính khả dụng cho các chức năng cốt lõi, không tập trung vào các yếu tố tùy biến phức tạp hay tối ưu hóa trải nghiệm người dùng ở mức độ cao.
- **Quản lý người dùng:** Hệ thống được phát triển dưới dạng ứng dụng cho một người dùng (single-tenant), chưa bao gồm các tính năng xác thực, phân quyền hay quản lý đa người dùng.
- **Cơ chế học tập:** Mô hình không có khả năng tự học hay cải thiện ([online learning](#)) từ các tương tác của người dùng. Việc cập nhật tri thức hoàn toàn phụ thuộc vào việc người dùng cung cấp tài liệu mới.
- **Tối ưu hóa hiệu năng:** Hiệu năng của hệ thống phụ thuộc vào tài nguyên phần cứng (CPU/GPU) và chưa được tối ưu hóa để vận hành ở quy mô lớn với lượng truy cập đồng thời cao.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN CÔNG NGHỆ

Để xây dựng một hệ thống Chatbot vững chắc và hiệu quả, việc nắm vững nền tảng lý thuyết và lựa chọn ngăn xếp công nghệ phù hợp là yếu tố tiên quyết. Chương này sẽ trình bày các khái niệm khoa học cốt lõi làm nền tảng cho dự án và phân tích chi tiết các công nghệ đã được lựa chọn để hiện thực hóa hệ thống.

2.1 Cơ sở Lý thuyết

2.1.1 Mô hình Ngôn ngữ Lớn (Large Language Models - LLM)

Mô hình Ngôn ngữ Lớn (LLM) không phải là một công nghệ đơn lẻ mà là đỉnh cao của một quá trình tiến hóa kéo dài nhiều năm trong lĩnh vực Xử lý Ngôn ngữ Tự nhiên. Để hiểu rõ cơ chế hoạt động và các giới hạn của chúng, cần phải phân tích các cột mốc lý thuyết nền tảng đã định hình nên chúng.

2.1.1.1 Kiến trúc Transformer: Cuộc cách mạng về xử lý chuỗi

Cột mốc quan trọng nhất, được xem là đã khai sinh ra kỷ nguyên LLM hiện đại, là sự ra đời của kiến trúc [Transformer](#) được công bố bởi Google vào năm 2017.

Trước Transformer, các mô hình xử lý chuỗi như Mạng nơ-ron hồi quy (RNN) và Long Short-Term Memory (LSTM) là kiến trúc chủ đạo. Tuy nhiên, chúng có hai hạn chế lớn:

- (1) Khó khăn trong việc nắm bắt các phụ thuộc ngữ nghĩa ở khoảng cách xa (long-range dependencies) do vấn đề suy giảm/bùng nổ gradient;
- (2) Bản chất tuần tự (sequential) của chúng khiến việc xử lý không thể song song hóa, gây cản trở cho việc huấn luyện trên các tập dữ liệu cực lớn.

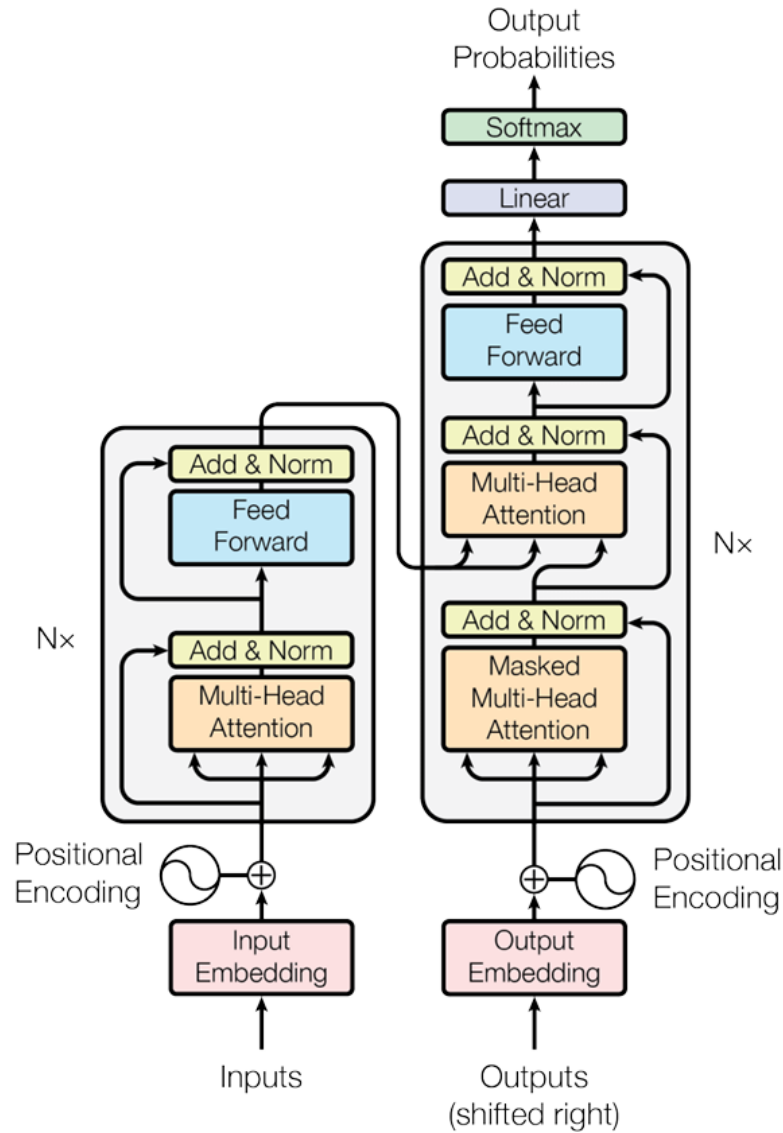
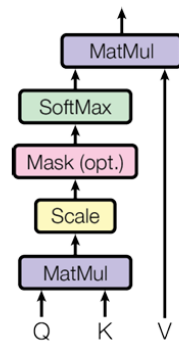


Figure 2.1: Kiến trúc Transformer

Kiến trúc Transformer đã giải quyết các vấn đề này bằng một cơ chế đột phá gọi là "Tự chú ý" ([Self-Attention](#)). Thay vì xử lý tuần tự, cơ chế này cho phép mô hình tính toán một biểu diễn cho mỗi từ trong câu bằng cách cân nhắc và "chú ý" đến tất cả các từ khác trong cùng một câu, bất kể vị trí của chúng. Về mặt khái niệm, với mỗi từ, cơ chế self-attention sẽ tính toán ba vector:

- **Query (Q):** Đại diện cho từ hiện tại, dùng để "hỏi" các từ khác.
- **Key (K):** Đại diện cho các từ khác trong chuỗi, dùng để được so khớp với Query.
- **Value (V):** Đại diện cho nội dung thực sự của các từ khác.

Scaled Dot-Product Attention



Multi-Head Attention

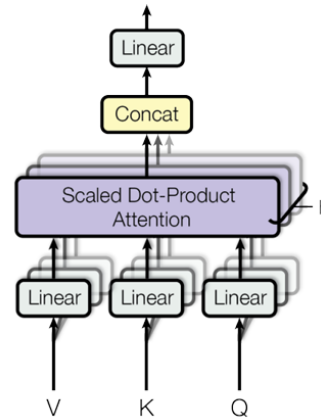


Figure 2.2: (Trái) Scaled Dot-Product Attention. (Phải) Multi-Head Attention.

Độ tương quan giữa vector Query của một từ với vector Key của các từ khác sẽ quyết định "trọng số chú ý" mà mô hình dành cho các từ đó. Biểu diễn cuối cùng của từ hiện tại sẽ là tổng có trọng số của các vector Value. Khả năng xử lý song song và nắm bắt ngữ cảnh toàn cục này đã mở đường cho việc xây dựng các mô hình với hàng tỷ tham số.

2.1.1.2 Các Mô hình Tiền huấn luyện: BERT và GPT

Sau khi Transformer ra đời, cộng đồng nghiên cứu đã nhanh chóng phát triển các mô hình khổng lồ được tiền huấn luyện (**pre-trained**) trên dữ liệu văn bản thô. Hai hướng tiếp cận chính đã định hình nên thế hệ LLM đầu tiên:

1. **Mô hình Tự mã hóa (Auto-Encoding):** Mô hình tiêu biểu của hướng tiếp cận này là **BERT (Bidirectional Encoder Representations from Transformers)** sử dụng phần mã hóa (**Encoder**) của kiến trúc Transformer. Mục tiêu của **BERT** là học các biểu diễn ngôn ngữ sâu sắc bằng cách huấn luyện với nhiệm vụ "Mô hình hóa Ngôn ngữ bị che" (**Masked Language Modeling - MLM**). Trong đó, một vài từ trong câu đầu vào sẽ bị che đi (mask), và mô hình phải dự đoán các từ bị che dựa trên ngữ cảnh xung quanh (cả bên trái và bên phải). Khả năng "nhìn" ngữ cảnh hai chiều này giúp **BERT** cực kỳ hiệu quả trong các tác vụ hiểu ngôn ngữ (**NLU**) như phân loại tình cảm, nhận dạng thực thể, trả lời câu hỏi trích xuất.
2. **Mô hình Tự hồi quy (Autoregressive):** Mô hình tiêu biểu của hướng tiếp cận này là **GPT** được tiên phong bởi OpenAI. Dòng mô hình **GPT (Generative Pre-trained Transformer)** sử dụng phần giải mã (**Decoder**) của kiến trúc Transformer. **GPT** được huấn luyện với một mục tiêu đơn giản hơn: dự đoán từ tiếp theo trong một chuỗi văn bản dựa trên tất cả các từ đã có trước đó. Bản chất tự hồi quy, một chiều (unidirectional) này làm cho **GPT** trở nên xuất sắc trong các tác vụ tạo sinh ngôn ngữ (**NLG**) như viết tiếp câu chuyện, tóm tắt văn bản, dịch máy và sinh mã nguồn.

2.1.1.3 Luật Quy mô và các Khả năng Bất ngờ (Emergent Abilities)

Một trong những khám phá quan trọng nhất thúc đẩy cuộc đua LLM là "Luật Quy mô" ([Scaling Laws](#)), được hệ thống hóa trong bài báo được công bố bởi [OpenAI](#). Nghiên cứu này chỉ ra rằng hiệu năng của mô hình (đo bằng loss trên tập validation) có thể được dự đoán một cách chính xác khi tăng quy mô của ba yếu tố: số lượng tham số (model size), kích thước tập dữ liệu (dataset size) và năng lực tính toán (compute).

Luật này đã tạo ra một lộ trình rõ ràng: để có mô hình tốt hơn, hãy làm cho nó lớn hơn. Khi các mô hình như [GPT-3](#) đạt đến quy mô hàng trăm tỷ tham số, các nhà nghiên cứu đã quan sát thấy một hiện tượng thú vị gọi là "Khả năng Bất ngờ" ([Emergent Abilities](#)). Đây là những khả năng mà mô hình nhỏ không có, nhưng lại đột ngột xuất hiện ở các mô hình có quy mô đủ lớn, ví dụ như khả năng thực hiện các phép toán số học, suy luận logic nhiều bước, hay học trong ngữ cảnh ([in-context learning](#)) chỉ qua vài ví dụ.

2.1.1.4 Căn chỉnh với Ý định của Con người: RLHF

Một mô hình được tiền huấn luyện đơn thuần chỉ là một cỗ máy dự đoán từ tiếp theo. Để nó trở thành một trợ lý hữu ích, có khả năng tuân theo chỉ dẫn và đưa ra câu trả lời an toàn, một bước căn chỉnh (alignment) là cần thiết. Kỹ thuật đột phá cho việc này là Học tăng cường từ Phản hồi của Con người (**Reinforcement Learning from Human Feedback - RLHF**), được mô tả chi tiết trong bài báo về [InstructGPT](#), tiền thân của [ChatGPT](#).

Quy trình [RLHF](#) thường bao gồm ba bước:

1. **Giám sát và Tinh chỉnh (Supervised Fine-Tuning):** Thu thập một tập dữ liệu nhỏ gồm các cặp (prompt, câu trả lời mẫu) do con người viết. Tinh chỉnh mô hình LLM trên tập dữ liệu này để nó học cách tuân theo chỉ dẫn cơ bản.
2. **Huấn luyện Mô hình Phần thưởng (Reward Model):** Cho mô hình đã tinh chỉnh ở bước 1 tạo ra nhiều câu trả lời cho cùng một [prompt](#). Con người sẽ xếp hạng các câu trả lời này từ tốt nhất đến tệ nhất. Dữ liệu so sánh này được dùng để huấn luyện một mô hình riêng, gọi là Mô hình Phần thưởng, có nhiệm vụ chấm điểm (đưa ra một phần thưởng) cho bất kỳ câu trả lời nào của LLM.
3. **Tối ưu hóa bằng Học tăng cường (Reinforcement Learning):** Sử dụng các thuật toán học tăng cường (như [PPO - Proximal Policy Optimization](#)) để tiếp tục tinh chỉnh mô hình LLM. Trong bước này, mô hình sẽ tự tạo ra câu trả lời và nhận "phần thưởng" từ Mô hình Phần thưởng. Mục tiêu của nó là tối đa hóa phần thưởng nhận được, qua đó học cách tạo ra các câu trả lời mà con người ưa thích.

LLM được sử dụng là một sản phẩm kế thừa từ tất cả những đột phá trên. Tuy nhiên, dù đã được căn chỉnh, bản chất xác suất và giới hạn kiến thức nội tại vẫn là những thách thức cố hữu

2.1.1.5 Các Thách thức và Giới hạn Cố hữu của LLM

Mặc dù đã đạt được những thành tựu vượt bậc, các Mô hình Ngôn ngữ Lớn vẫn tồn tại những thách thức và giới hạn mang tính cố hữu, xuất phát trực tiếp từ kiến trúc và phương pháp huấn luyện của chúng. Việc nhận diện rõ các giới hạn này là tiền đề quan trọng để đề xuất các giải pháp khắc phục.

- **Hiện tượng "ảo giác" (Hallucination):** Đây là một trong những thách thức nghiêm trọng nhất. "Ảo giác" là hiện tượng mô hình tạo ra các thông tin không chính xác, không có căn cứ hoặc hoàn toàn bịa đặt, nhưng lại được trình bày một cách tự tin và mạch lạc. Nguyên nhân cốt lõi của hiện tượng này đến từ các yếu tố sau:
 - **Bản chất xác suất:** LLM được tối ưu hóa để dự đoán từ tiếp theo có khả năng xuất hiện cao nhất về mặt thống kê, nhằm tạo ra văn bản trôi chảy và hợp lý. Mục tiêu này không đồng nhất với việc đảm bảo tính đúng đắn của thông tin. Mô hình có thể kết hợp các thực thể và sự kiện một cách sai lệch nếu sự kết hợp đó có xác suất cao trong không gian dữ liệu huấn luyện.
 - **Kiến thức mã hóa trong tham số (Parametric Knowledge):** Toàn bộ kiến thức của LLM được "nén" vào các trọng số của mạng nơ-ron. Đây là một dạng bộ nhớ ngầm, không thể cập nhật hay kiểm chứng một cách trực tiếp. Khi truy xuất kiến thức, mô hình phải "suy diễn" từ các trọng số này, một quá trình dễ dẫn đến sai sót và bịa đặt, đặc biệt với các thông tin ít xuất hiện trong dữ liệu huấn luyện.
 - **Dữ liệu huấn luyện nhiễu:** Dữ liệu từ Internet chứa đầy thông tin sai lệch, mâu thuẫn và thiên kiến. LLM học từ tất cả các nguồn này mà không có cơ chế kiểm chứng, do đó có thể tái tạo lại những thông tin không chính xác.

Trong thực tế, kể cả những mô hình tối tân nhất cũng không thể tránh được hiện tượng "ảo giác".

Dataset	Metric	o3	o4-mini	o1
SimpleQA	accuracy (higher is better)	0.49	0.20	0.47
	hallucination rate (lower is better)	0.51	0.79	0.44
PersonQA	accuracy (higher is better)	0.59	0.36	0.47
	hallucination rate (lower is better)	0.33	0.48	0.16

Figure 2.3: Hiện tượng ảo giác của model o3 và o4-mini

Sự "thông minh" của AI không đồng nghĩa với sự chính xác. Ngay cả những mô hình "tối tân" nhất hiện nay, như các mô hình thuộc dòng GPT-4, cũng không thể tránh khỏi ảo giác ([hallucination](#)).

Điều đáng nói là: *Hallucination không chỉ xảy ra trong các câu hỏi phức tạp, mà có thể xảy ra thường xuyên nhất ở những câu hỏi đơn giản nhất.*

- **Giới hạn kiến thức (Knowledge Cutoff):** Quá trình huấn luyện một LLM quy mô lớn đòi hỏi chi phí tính toán và thời gian khổng lồ. Do đó, một khi đã được huấn luyện, kiến thức của mô hình sẽ bị "đóng băng" tại thời điểm dữ liệu huấn luyện được thu thập (gọi là "knowledge cutoff date"). Điều này dẫn đến hai hệ quả trực tiếp:
 - **Thông tin lỗi thời:** Mô hình hoàn toàn không biết về các sự kiện, phát minh, hay tin tức xảy ra sau thời điểm giới hạn kiến thức.
 - **Không thể truy cập thông tin riêng tư:** Mô hình không thể tiếp cận các nguồn dữ liệu độc quyền, nội bộ của một tổ chức, doanh nghiệp hoặc các tài liệu cá nhân của người dùng.
- **Thiếu tính diễn giải (Lack of Interpretability):** Do kiến trúc mạng nơ-ron sâu với hàng tỷ tham số, quá trình "suy nghĩ" để đưa ra một câu trả lời của LLM là một "hộp đen" (black box). Chúng ta không thể dễ dàng truy vết được tại sao mô hình lại tạo ra một câu trả lời cụ thể, hay nó đã dựa vào phần nào của kiến thức nội tại để suy luận. Sự thiếu minh bạch này gây khó khăn trong việc gỡ lỗi và tạo ra rào cản về niềm tin khi ứng dụng trong các lĩnh vực yêu cầu tính giải trình cao như y tế hay tài chính.

Những thách thức trên cho thấy, việc chỉ dựa vào LLM như một thực thể độc lập là không đủ để xây dựng các ứng dụng đáng tin cậy và có tính thực tiễn cao. Chính vì vậy, các kiến trúc bổ trợ như Retrieval-Augmented Generation (RAG) được đề xuất như một giải pháp chiến lược để khắc phục các giới hạn này, đặc biệt là vấn đề về "ảo giác" và kiến thức lỗi thời.

2.1.2 Retrieval-Augmented Generation (RAG)

Để giải quyết các thách thức cố hữu của LLM đã được phân tích ở mục trước (hiện tượng "ảo giác", kiến thức lỗi thời và thiếu tính diễn giải), đề tài áp dụng kiến trúc **Retrieval-Augmented Generation (RAG)**. RAG là một kiến trúc lai, kết hợp sức mạnh của các mô hình truy xuất thông tin (retrieval models) và mô hình sinh ngôn ngữ (generative-models).

Thay vì chỉ dựa vào kiến thức nội tại (parametric knowledge) đã được mã hóa trong các tham số, hệ thống RAG có khả năng truy cập và sử dụng một cơ sở tri thức bên ngoài (non-parametric knowledge) để tạo ra câu trả lời. Quá trình này về cơ bản bao gồm hai giai đoạn chính: lập chỉ mục (indexing) và truy xuất-tạo sinh (retrieval-generation).

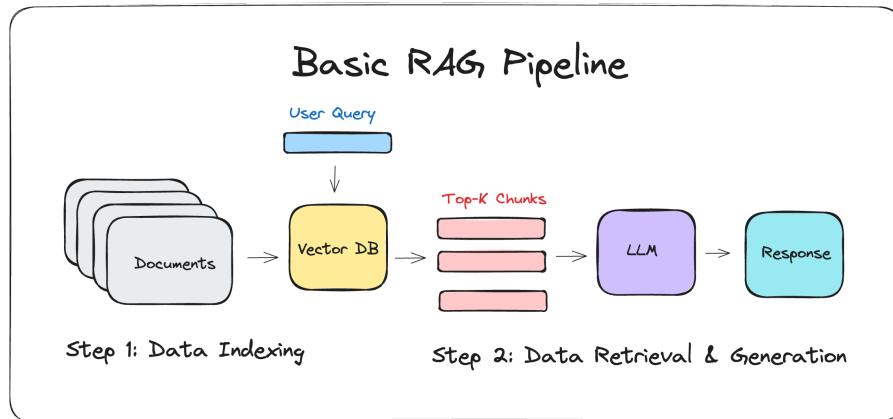


Figure 2.4: RAG Pipeline

2.1.2.1 Giai đoạn 1: Xây dựng và Lập chỉ mục Cơ sở tri thức (Indexing Phase)

Đây là một quá trình xử lý "offline", được thực hiện trước khi người dùng tương tác với hệ thống, nhằm chuẩn bị nguồn kiến thức cho việc truy xuất. Giai đoạn này bao gồm các bước sau:

1. **Tải Dữ liệu (Load):** Hệ thống tiếp nhận các tài liệu nguồn từ người dùng (ví dụ: các tệp PDF, DOCX, TXT, hoặc nội dung từ các trang web) và nạp chúng vào bộ nhớ.
2. **Phân mảnh Văn bản (Chunking/Splitting):** Các tài liệu lớn được chia thành các đoạn văn bản nhỏ hơn, gọi là các "chunk". Việc phân mảnh này rất quan trọng vì hai lý do:
 - (1) Nó giúp việc truy xuất trở nên chính xác hơn, vì các chunk nhỏ sẽ tập trung vào một chủ đề cụ thể;
 - (2) Nó đảm bảo các đoạn văn bản truy xuất được có thể vừa với giới hạn ngữ cảnh (context window) của LLM.
3. **Vector hóa (Embedding):** Mỗi chunk văn bản sau đó được đưa qua một mô hình embedding (ví dụ: Sentence-BERT, text-embedding-ada-002). Mô hình này sẽ mã hóa ngữ nghĩa của chunk thành một vector số thực đa chiều (embedding vector). Các chunk có nội dung ngữ nghĩa tương tự nhau sẽ có các vector biểu diễn nằm gần nhau trong không gian vector.
4. **Lưu trữ và Lập chỉ mục (Store & Index):** Tất cả các cặp (vector, nội dung chunk gốc) sẽ được lưu trữ và lập chỉ mục trong một cơ sở dữ liệu vector (Vector Database). Cơ sở dữ liệu này được tối ưu hóa cho việc tìm kiếm tương đồng (similarity search) với tốc độ cao, cho phép tìm ra các vector gần nhất với một vector truy vấn cho trước một cách hiệu quả.

Kết thúc giai đoạn này, chúng ta có một cơ sở tri thức đã được số hóa và sẵn sàng cho việc truy vấn.

2.1.2.2 Giai đoạn 2: Truy xuất và Tạo sinh (Retrieval and Generation Phase)

Đây là quá trình "online", diễn ra trong thời gian thực khi người dùng đặt câu hỏi.

1. **Nhận Truy vấn (User Query):** Người dùng nhập một câu hỏi bằng ngôn ngữ tự nhiên.
2. **Vector hóa Truy vấn:** Câu hỏi của người dùng cũng được đưa qua cùng một mô hình embedding đã sử dụng ở Giai đoạn 1 để tạo ra một vector truy vấn. Việc sử dụng chung một mô hình đảm bảo rằng vector của câu hỏi và vector của các chunk tài liệu nằm trong cùng một không gian ngữ nghĩa.
3. **Tìm kiếm Tương đồng (Similarity Search):** Hệ thống sử dụng vector truy vấn để tìm kiếm trong cơ sở dữ liệu vector. Thuật toán tìm kiếm (ví dụ: tìm kiếm lân cận gần nhất - ANN) sẽ trả về k chunk tài liệu có vector gần nhất (tương đồng nhất về mặt ngữ nghĩa) với vector truy vấn. Các chunk này được xem là ngữ cảnh liên quan nhất để trả lời câu hỏi.
4. **Tăng cường Prompt (Prompt Augmentation):** Các chunk văn bản vừa được truy xuất sẽ được kết hợp với câu hỏi gốc của người dùng để tạo thành một câu lệnh đầu vào (prompt) hoàn chỉnh. Prompt này thường có cấu trúc như sau:

Dựa vào ngữ cảnh dưới đây:

[Nội dung của chunk 1]

[Nội dung của chunk 2]

...

Hãy trả lời câu hỏi sau: [Câu hỏi gốc của người dùng]

5. **Tạo sinh Câu trả lời (Generate):** Prompt đã được tăng cường này cuối cùng được gửi đến LLM. Bằng cách cung cấp ngữ cảnh rõ ràng, chúng ta "hướng dẫn" LLM tạo ra câu trả lời dựa trên các bằng chứng xác thực đã được tìm thấy, thay vì dựa vào kiến thức nội tại có thể đã lỗi thời hoặc không chính xác của nó.

2.1.2.3 Ưu điểm của Kiến trúc RAG

So với việc sử dụng LLM một cách độc lập, kiến trúc RAG mang lại nhiều ưu điểm vượt trội:

- **Tăng tính chính xác, giảm thiểu "ảo giác":** Bằng cách "neo" câu trả lời vào các nguồn thông tin xác thực, RAG làm giảm đáng kể khả năng LLM bịa đặt thông tin.
- **Cập nhật kiến thức linh hoạt và tiết kiệm chi phí:** Thay vì phải huấn luyện lại (retrain) hoặc tinh chỉnh (fine-tune) toàn bộ LLM – một quá trình cực kỳ tốn kém – chúng ta chỉ cần cập nhật cơ sở dữ liệu vector bằng cách thêm, xóa hoặc sửa các tài liệu.

- **Tăng tính minh bạch và có thể kiểm chứng:** Hệ thống có thể trích dẫn các nguồn tài liệu (các chunk đã được truy xuất) mà nó đã sử dụng để tạo ra câu trả lời. Điều này cho phép người dùng tự kiểm chứng thông tin, qua đó tăng cường sự tin tưởng và giải quyết một phần vấn đề "hộp đen" của LLM.

2.1.3 Vector Embeddings và Cơ sở dữ liệu Vector

Để giai đoạn truy xuất thông tin (Retrieval) trong kiến trúc RAG có thể hoạt động hiệu quả, hai công nghệ nền tảng là Vector Embeddings và Cơ sở dữ liệu Vector đóng vai trò then chốt. Chúng phối hợp với nhau để chuyển đổi ngôn ngữ tự nhiên thành một dạng mà máy tính có thể so sánh và tìm kiếm với tốc độ cao.

2.1.3.1 Vector Embeddings: Mã hóa Ngữ nghĩa thành Dạng Toán học

Vector Embedding là quá trình chuyển đổi một đơn vị dữ liệu phi cấu trúc (như một từ, một câu, hoặc một đoạn văn bản) thành một vector số thực trong không gian đa chiều. Quá trình này được thực hiện bởi các mô hình embedding, vốn là các mạng nơ-ron được huấn luyện trên một lượng lớn dữ liệu văn bản.

- **Mục tiêu:** Mục tiêu chính của mô hình embedding là nắm bắt được "ngữ nghĩa" của văn bản. Sau khi được huấn luyện, mô hình có khả năng tạo ra các vector sao cho các đoạn văn bản có ý nghĩa tương tự nhau sẽ có các vector biểu diễn nằm gần nhau trong không gian vector. Ví dụ, vector của câu "Chi phí sinh hoạt ở Hà Nội là bao nhiêu?" sẽ nằm rất gần với vector của câu "Giá cả cuộc sống tại thủ đô như thế nào?".
- **Đo lường sự tương đồng:** Sự "gần nhau" trong không gian vector thường được đo bằng các độ đo tương đồng. Độ đo phổ biến nhất là Độ tương đồng Cosine (Cosine Similarity), tính toán giá trị cosine của góc giữa hai vector.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}||\mathbf{B}|} \quad (2.1)$$

Giá trị này dao động từ -1 đến 1. Một giá trị gần bằng 1 cho thấy hai vector gần như cùng hướng, tức là hai đoạn văn bản có ngữ nghĩa rất tương đồng. Ngược lại, giá trị gần bằng 0 hoặc -1 cho thấy chúng khác xa nhau về mặt ngữ nghĩa.

- **Vai trò trong RAG:** Trong dự án này, Vector Embeddings đóng vai trò là "ngôn ngữ chung" để so sánh câu hỏi của người dùng và nội dung của các tài liệu. Bằng cách mã hóa cả hai thành vector, hệ thống có thể thực hiện tìm kiếm dựa trên ngữ nghĩa thay vì chỉ dựa vào việc so khớp từ khóa (keyword matching) một cách máy móc.

2.1.3.2 Cơ sở dữ liệu Vector: Lưu trữ và Truy xuất Hiệu quả

Sau khi các đoạn văn bản đã được chuyển đổi thành vector, chúng ta cần một hệ thống chuyên dụng để lưu trữ và tìm kiếm trên hàng ngàn, thậm chí hàng triệu

vector này một cách hiệu quả. Đây chính là vai trò của Cơ sở dữ liệu Vector.

- **Thách thức của tìm kiếm trong không gian đa chiều:** Việc tìm kiếm vector gần nhất một cách chính xác (brute-force search) bằng cách so sánh vector truy vấn với tất cả các vector trong cơ sở dữ liệu sẽ trở nên cực kỳ chậm chạp và tốn kém tài nguyên khi số lượng vector tăng lên. Đây là một biểu hiện của "lời nguyền của số chiều" (curse of dimensionality).
- **Giải pháp - Tìm kiếm Lân cận Gần nhất Xấp xỉ (ANN):** Để giải quyết vấn đề này, các cơ sở dữ liệu vector hiện đại sử dụng các thuật toán Tìm kiếm Lân cận Gần nhất Xấp xỉ (Approximate Nearest Neighbor - ANN). Thay vì đảm bảo tìm ra kết quả chính xác 100%, các thuật toán này chấp nhận một sai số nhỏ để đổi lấy sự cải thiện vượt bậc về tốc độ tìm kiếm (thường nhanh hơn hàng trăm đến hàng nghìn lần). Đối với các ứng dụng như RAG, việc tìm ra một tập hợp các kết quả "gần như tốt nhất" là hoàn toàn đủ để cung cấp ngữ cảnh chất lượng cho LLM.
- **FAISS - Lựa chọn cho dự án:** Trong dự án này, **FAISS** (Facebook AI Similarity Search) được lựa chọn.
 - **Bản chất:** **FAISS** là một thư viện mã nguồn mở, hiệu năng cực cao, được phát triển bởi Meta AI. Nó cung cấp một bộ sưu tập phong phú các thuật toán để lập chỉ mục (indexing) và tìm kiếm vector.
 - **Lý do lựa chọn:** Mặc dù không phải là một hệ quản trị cơ sở dữ liệu hoàn chỉnh (không có các tính năng như API mạng, quản lý người dùng), **FAISS** rất phù hợp cho dự án này vì:
 - (1) Tốc độ vượt trội: Nó được tối ưu hóa sâu sắc bằng C++ và có thể tận dụng GPU để tăng tốc tìm kiếm.
 - (2) Linh hoạt: Cung cấp nhiều loại chỉ mục khác nhau (như **IndexFlatL2**, **IndexIVFFlat**) cho phép nhà phát triển cân bằng giữa tốc độ, độ chính xác và dung lượng bộ nhớ.
 - (3) Tích hợp dễ dàng: Hoạt động như một thư viện Python, chạy trực tiếp trên bộ nhớ (in-memory), giúp đơn giản hóa kiến trúc hệ thống, không cần triển khai một dịch vụ cơ sở dữ liệu riêng biệt.

Sự kết hợp giữa các mô hình embedding mạnh mẽ và thư viện tìm kiếm vector hiệu quả như **FAISS** tạo thành xương sống kỹ thuật vững chắc cho giai đoạn truy xuất của hệ thống Chatbot.

2.2 Ngăn xếp Công nghệ (Technology Stack)

Việc lựa chọn một ngăn xếp công nghệ phù hợp là một quyết định kỹ thuật then chốt, ảnh hưởng trực tiếp đến hiệu quả phát triển, khả năng bảo trì và hiệu năng của sản phẩm cuối cùng. Dựa trên các yêu cầu của một hệ thống Chatbot RAG, dự án đã lựa chọn một ngăn xếp công nghệ hiện đại, hiệu quả và có hệ sinh thái hỗ trợ mạnh mẽ.

2.2.1 Phát triển Backend

Backend là nơi xử lý toàn bộ logic nghiệp vụ cốt lõi, từ việc tương tác với các mô hình AI đến quản lý dữ liệu.

- **Ngôn ngữ:** [Python](#)

Lý do lựa chọn: [Python](#) là ngôn ngữ thống trị trong lĩnh vực Khoa học Dữ liệu và Trí tuệ Nhân tạo. Sự lựa chọn này được củng cố bởi một hệ sinh thái thư viện mạnh mẽ, cho phép xây dựng các thành phần của pipeline RAG một cách trực tiếp và linh hoạt thay vì phụ thuộc vào một framework lớn duy nhất. Các thư viện chính được sử dụng bao gồm:

- [HuggingFaceTransformers](#) & [Sentence-Transformers](#): Cung cấp quyền truy cập vào hàng ngàn mô hình AI đã được tiền huấn luyện. Trong dự án, thư viện này được dùng để tích hợp mô hình embedding. Việc sử dụng trực tiếp thư viện cho phép kiểm soát sâu hơn quá trình vector hóa và lựa chọn mô hình phù hợp nhất.
- [FAISS](#): Là thư viện hiệu năng cao từ Meta AI, được sử dụng để xây dựng và truy vấn cơ sở dữ liệu vector. Việc tích hợp [FAISS](#) trực tiếp giúp tối ưu hóa tốc độ tìm kiếm tương đồng ngữ nghĩa, một tác vụ cốt lõi của RAG.

- **Framework:** [FastAPI](#)

Lý do lựa chọn: [FastAPI](#) được chọn để xây dựng các API của hệ thống nhờ các ưu điểm vượt trội về hiệu năng và trải nghiệm phát triển:

- **Hiệu năng cao và Lập trình Bất đồng bộ:** Hỗ trợ [async/await](#) một cách tự nhiên, [FastAPI](#) rất phù hợp để xử lý các tác vụ I/O-bound kéo dài (như gọi API đến LLM, xử lý file lớn) mà không làm block luồng chính, đảm bảo khả năng đáp ứng của hệ thống khi có nhiều người dùng đồng thời.
- **Kiểm tra dữ liệu chặt chẽ:** Tích hợp sẵn [Pydantic](#), [FastAPI](#) thực thi việc kiểm tra kiểu dữ liệu một cách nghiêm ngặt, tạo ra một "hợp đồng" dữ liệu rõ ràng giữa frontend và backend, giảm thiểu lỗi runtime và tăng tính ổn định.
- **Tự động sinh tài liệu API:** Khả năng tự động tạo ra tài liệu API tương tác (thông qua [SwaggerUI](#) và [ReDoc](#)) giúp đơn giản hóa quá trình kiểm thử, tích hợp và bảo trì API.

2.2.1.1 Mô hình Embedding: Sentence-BERT

Trong dự án này, mô hình **Sentence-BERT** ([all-MiniLM-L6-v2](#)) được sử dụng để thực hiện việc vector hóa văn bản. Đây là một lựa chọn kỹ thuật quan trọng dựa trên các tiêu chí về hiệu năng, chất lượng và tính khả thi.

- **Kiến trúc nền tảng:** Được xây dựng trên kiến trúc BERT nhưng được tối ưu hóa đặc biệt cho việc tạo ra các embedding ở mức câu (sentence-level). Khác

với BERT gốc chỉ tạo ra embedding tốt ở mức token, Sentence-BERT được huấn luyện với nhiệm vụ đặc biệt để tạo ra các vector có ý nghĩa ở mức câu hoặc đoạn văn.

- **Kích thước vector:** Mô hình tạo ra các vector 384 chiều, cân bằng tốt giữa khả năng biểu diễn thông tin và hiệu quả tính toán. Vector có số chiều thấp hơn giúp tiết kiệm bộ nhớ và tăng tốc độ tìm kiếm trong FAISS.
- **Tốc độ xử lý:** Với kiến trúc "Mini" (6 layers), mô hình này được tối ưu hóa để cân bằng giữa chất lượng embedding và tốc độ xử lý, phù hợp cho việc xử lý thời gian thực trong ứng dụng chatbot.
- **Hỗ trợ đa ngôn ngữ:** Mặc dù được huấn luyện chủ yếu trên tiếng Anh, mô hình vẫn có khả năng xử lý tiếng Việt ở mức độ chấp nhận được, phù hợp với yêu cầu của dự án.

2.2.1.2 Tích hợp Google Custom Search API để mở rộng tri thức

Để khắc phục giới hạn kiến thức của các tài liệu nội bộ, hệ thống được tích hợp với **Google Custom Search API**, cho phép truy xuất thông tin thời gian thực từ Internet.

- **API Endpoint:** Sử dụng Google Custom Search JSON API v1, cung cấp khả năng tìm kiếm có cấu trúc và kiểm soát được.
- **Tham số tìm kiếm chính:**
 - `num_results`: Số lượng kết quả trả về (mặc định 4 để tối ưu thời gian xử lý)
 - `dateRestrict`: Lọc theo thời gian (d1: 1 ngày, w1: 1 tuần, m1: 1 tháng, y1: 1 năm)
 - `siteSearch`: Giới hạn tìm kiếm trong một trang web cụ thể
 - `safe=active`: Kích hoạt SafeSearch để lọc nội dung không phù hợp
- **Xử lý lỗi và Fallback:** Hệ thống được thiết kế với cơ chế xử lý lỗi mạnh mẽ, bao gồm:
 - Xử lý lỗi 403 (hết quota hoặc API key không hợp lệ)
 - Timeout handling (10 giây)
 - Fallback results khi API không khả dụng

2.2.1.3 Tích hợp vào Pipeline RAG

Google Search hoạt động song song với RAG truyền thống:

1. **Query Analysis:** Hệ thống phân tích câu hỏi của người dùng để quyết định có cần tìm kiếm web hay không.

2. **Parallel Processing:** Nếu cần thiết, tìm kiếm web được thực hiện song song với truy xuất từ cơ sở dữ liệu vector nội bộ.
3. **Result Fusion:** Kết quả từ cả hai nguồn được tổng hợp và xếp hạng để cung cấp ngữ cảnh tốt nhất cho LLM.

Sự kết hợp này tạo ra một hệ thống RAG "lai" (Hybrid RAG) có khả năng trả lời cả những câu hỏi về thông tin nội bộ cụ thể và những câu hỏi về sự kiện, tin tức thời sự mới nhất.

2.2.2 Phát triển Frontend

Frontend chịu trách nhiệm mang lại một trải nghiệm tương tác trực quan và thân thiện cho người dùng.

- **Ngôn ngữ và Thu viện:** [TypeScript](#) và [React](#)

Lý do lựa chọn: Sự kết hợp giữa [React](#) và [TypeScript](#) tạo ra một nền tảng vững chắc để xây dựng các giao diện người dùng phức tạp.

- [React](#): Với kiến trúc dựa trên component, [React](#) cho phép chia nhỏ giao diện thành các thành phần độc lập, có thể tái sử dụng. Điều này đặc biệt hữu ích để quản lý các trạng thái phức tạp của một ứng dụng chat, như lịch sử hội thoại, danh sách tài liệu, và các cài đặt của người dùng.
- [TypeScript](#): Bằng cách bổ sung hệ thống kiểu dữ liệu tĩnh cho [JavaScript](#), [TypeScript](#) giúp phát hiện lỗi ngay trong quá trình viết code, cải thiện khả năng tự động hoàn thành của trình soạn thảo và làm cho mã nguồn trở nên dễ đọc, dễ bảo trì hơn, đặc biệt quan trọng khi dự án phát triển.

- **Công cụ xây dựng (Build Tool):** [Vite.js](#)

Lý do lựa chọn: [Vite](#) được chọn để tối ưu hóa trải nghiệm phát triển (*Developer Experience*). Bằng cách tận dụng [native ESmodules](#) của trình duyệt, [Vite](#) cung cấp một máy chủ phát triển với tốc độ khởi động gần như tức thì và tính năng Hot Module Replacement (HMR) cực nhanh, giúp rút ngắn đáng kể chu kỳ phản hồi khi chỉnh sửa giao diện.

2.2.3 Đóng gói và Triển khai

- **Công cụ:** [Docker](#) và [DockerCompose](#)

Lý do lựa chọn: Giải pháp container hóa này được chọn để giải quyết bài toán về tính nhất quán và di động của môi trường.

- [Docker](#): Đóng gói từng dịch vụ (backend, frontend) vào các *container* cô lập, chứa đựng tất cả các thư viện và phụ thuộc cần thiết. Điều này đảm bảo sự đồng nhất về môi trường (*environmental parity*), loại bỏ hoàn toàn vấn đề "chạy được trên máy tôi nhưng không chạy trên máy khác".

- **DockerCompose**: Cho phép định nghĩa và điều phối một ứng dụng đa dịch vụ thông qua một tệp cấu hình **YAML** duy nhất. Với một lệnh, nhà phát triển có thể khởi chạy, kết nối và quản lý toàn bộ hệ thống, giúp đơn giản hóa triệt để quy trình cài đặt và triển khai.

Với nền tảng lý thuyết vững chắc và ngăn xếp công nghệ được lựa chọn cẩn thận, dự án có đủ cơ sở để tiến hành thiết kế và xây dựng hệ thống chi tiết, sẽ được trình bày ở chương tiếp theo.

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình chuyển đổi từ các yêu cầu nghiệp vụ sang một mô hình thiết kế hệ thống cụ thể. Nội dung bao gồm việc phân tích chi tiết các yêu cầu chức năng và phi chức năng, sau đó mô hình hóa hệ thống thông qua các sơ đồ UML và sơ đồ luồng dữ liệu để làm nền tảng cho việc triển khai ở các chương sau.

3.1 Phân tích Yêu cầu

3.1.1 Yêu cầu Chức năng (Functional Requirements)

Dựa trên mục tiêu của đề tài, các yêu cầu chức năng sau đây được xác định, thể hiện dưới dạng User Story để tập trung vào giá trị mang lại cho người dùng.

Table 3.1: Yêu cầu Chức năng (Functional Requirements)

ID	User Story	Chức năng tương ứng	Ghi chú
F-01	Là người dùng, tôi muốn tải lên các tệp tài liệu (.pdf, .docx, .txt) để xây dựng cơ sở tri thức cho Chatbot.	Chức năng Tải và Xử lý Tài liệu.	Giao diện cần có một khu vực cho phép kéo-thả hoặc chọn tệp từ máy tính.
F-02	Là người dùng, tôi muốn đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận được câu trả lời từ Chatbot.	Chức năng Hỏi-Đáp (Chat).	Giao diện cần có khung nhập liệu và khu vực hiển thị cuộc hội thoại.
F-03	Là người dùng, tôi muốn câu trả lời của Chatbot phải dựa trên nội dung tài liệu tôi đã cung cấp.	Luồng xử lý RAG từ cơ sở tri thức (VectorStore).	Đây là chức năng cốt lõi của hệ thống.
F-04	Là người dùng, tôi muốn Chatbot có thể tìm kiếm thông tin trên Internet nếu không tìm thấy trong tài liệu.	Tích hợp GoogleSearch vào luồng RAG .	Chức năng này có thể được bật/tắt bởi người dùng.
F-05	Là người dùng, tôi muốn xem lại lịch sử các cuộc hội thoại trước đó.	Chức năng Quản lý Hội thoại.	Hiển thị danh sách các cuộc hội thoại cũ, cho phép người dùng chọn và xem lại.
F-06	Là người dùng, tôi muốn xóa một cuộc hội thoại cũ để làm sạch giao diện.	Chức năng Xóa Hội thoại.	Cần có xác nhận trước khi xóa để tránh mất dữ liệu không mong muốn.
F-07	Là người dùng, tôi muốn có thể cấu hình một số tham số của hệ thống, ví dụ như chọn mô hình LLM .	Chức năng Cài đặt (Settings).	Giao diện cài đặt cho phép thay đổi các cấu hình cơ bản của hệ thống.
F-08	Là người dùng, tôi muốn bắt đầu một cuộc hội thoại mới bất cứ lúc nào.	Chức năng Tạo Hội thoại mới.	Nút bấm hoặc hành động để xóa ngữ cảnh hiện tại và bắt đầu lại.

3.1.2 Yêu cầu Phi chức năng (Non-functional Requirements)

Yêu cầu phi chức năng định nghĩa các tiêu chuẩn về chất lượng, hiệu năng và các đặc tính khác của hệ thống.

Table 3.2: Yêu cầu Phi chức năng (Non-functional Requirements)

ID	Thuộc tính	Mô tả Yêu cầu Cụ thể
NF-01	Hiệu năng	Thời gian phản hồi (từ lúc gửi câu hỏi đến ký tự đầu tiên) cần dưới 5 giây trong điều kiện thông thường. Tốc độ xử lý tài liệu phải hợp lý (ví dụ: tệp 10 trang xử lý dưới 30 giây).
NF-02	Tính khả dụng	Hệ thống phải hoạt động ổn định, không gặp lỗi nghiêm trọng trong quá trình sử dụng thông thường và demo.
NF-03	Tính dễ sử dụng	Giao diện người dùng phải trực quan, tối giản, cho phép người dùng thực hiện các chức năng chính (chat, tải file) mà không cần hướng dẫn phức tạp.
NF-04	Tính tương thích	Ứng dụng web phải hiển thị và hoạt động tốt trên các trình duyệt phổ biến phiên bản mới nhất (Chrome, Firefox, Edge).
NF-05	Bảo mật	Các thông tin nhạy cảm như khóa API phải được lưu trữ an toàn phía backend, không được phép lộ ra mã nguồn frontend.
NF-06	Tính bảo trì	Mã nguồn phải tuân thủ các nguyên tắc thiết kế tốt (ví dụ: SOLID), được cấu trúc thành các module rõ ràng, có tài liệu và comment đầy đủ để dễ dàng bảo trì và mở rộng.

3.2 Phân tích và Thiết kế hệ thống

3.2.1 Sơ đồ hoạt động

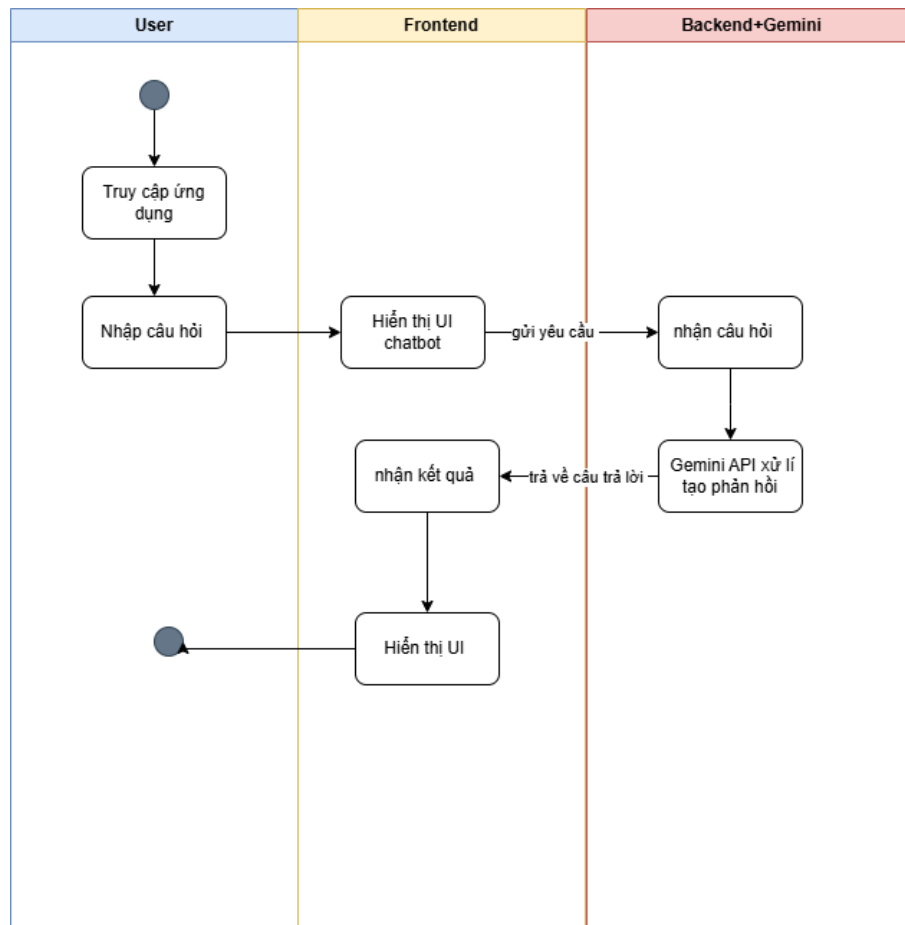


Figure 3.1: Sơ đồ Activity diagram mô tả tương tác người dùng với hệ thống

Sơ đồ trên mô tả trình tương tác giữa 3 chủ thể:

- Người
- Frontend
- Backend + Gemini AI

Khi người dùng truy cập ứng dụng, giao diện người dùng (UI) chatbot sẽ được hiển thị. Sau đó, người dùng nhập câu hỏi vào giao diện. Câu hỏi này được frontend xử lý và chuyển tiếp tới backend. Tại backend, hệ thống sẽ tiếp tục xử lý câu hỏi, tương tác với mô hình Gemini AI để tạo phản hồi phù hợp và trả lời lại cho người dùng thông qua giao diện.

3.2.2 Thiết kế Usecase

3.2.2.1 Mô tả Use Case chi tiết

Use Case	Mô tả
Upload Documents	Cho phép người dùng tải lên tài liệu (PDF, DOCX, TXT) để hệ thống phân tích và lưu trữ làm ngữ cảnh cho RAG.
Chat with Bot	Người dùng gửi câu hỏi và nhận phản hồi từ chatbot.
View Conversation History	Xem lại lịch sử các cuộc trò chuyện đã thực hiện trước đó.
Search Web	Kích hoạt chức năng tìm kiếm thông tin trên web để bổ sung cho RAG.
Manage API Keys	Quản lý các khóa API (Gemini, Google Search) để kết nối với dịch vụ bên ngoài.
Manage Settings	Cấu hình các tham số của hệ thống như mô hình LLM, chế độ tìm kiếm.

Table 3.4: Chi tiết các Use Case chính của hệ thống

3.2.2.2 Usecase tổng quát

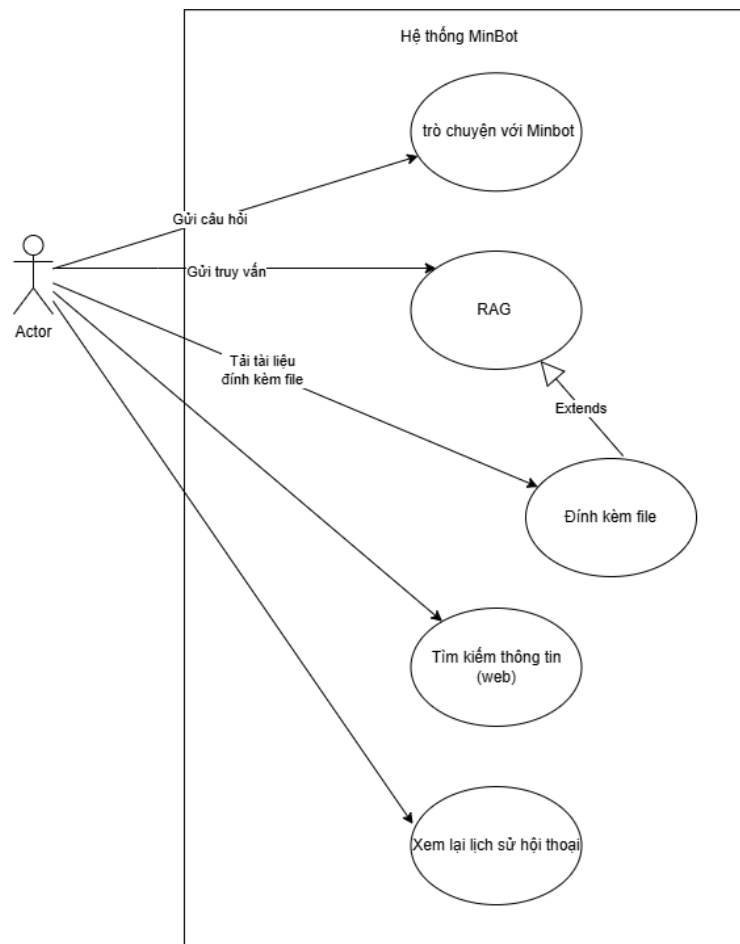


Figure 3.2: Sơ đồ Usecase tổng quát

Hệ thống bao gồm 4 chức năng chính và 1 tác nhân (actor):

- Trò chuyện với MinBot

- RAG + Đính kèm File
- Tìm kiếm thông tin (Web)
- Xem lại lịch sử

3.2.2.3 Usecase: Trò chuyện với MinBot

Mô tả chức năng: Người dùng nhập câu hỏi hoặc yêu cầu vào giao diện chat của MinBot. Sau đó, hệ thống sẽ gửi truy vấn này tới mô hình ngôn ngữ và trả lại phản hồi trực tiếp trên giao diện.

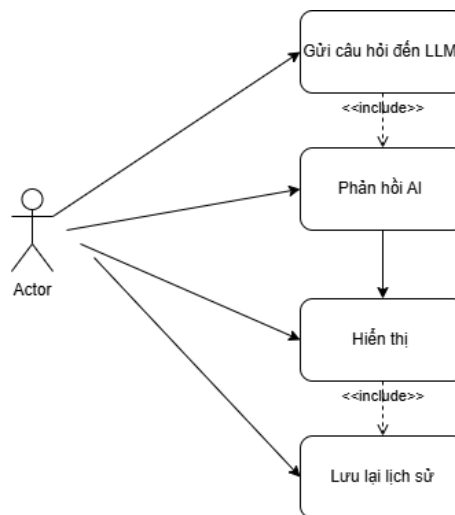


Figure 3.3: Sơ đồ Usecase trò chuyện với MinBot

3.2.2.4 Usecase: RAG

Mô tả chức năng: Người dùng tải lên file văn bản (PDF, DOCX...) để chatbot có thêm thông tin tham chiếu (Retrieval-Augmented Generation – RAG). Nội dung file được phân tích để hỗ trợ trả lời chính xác hơn.

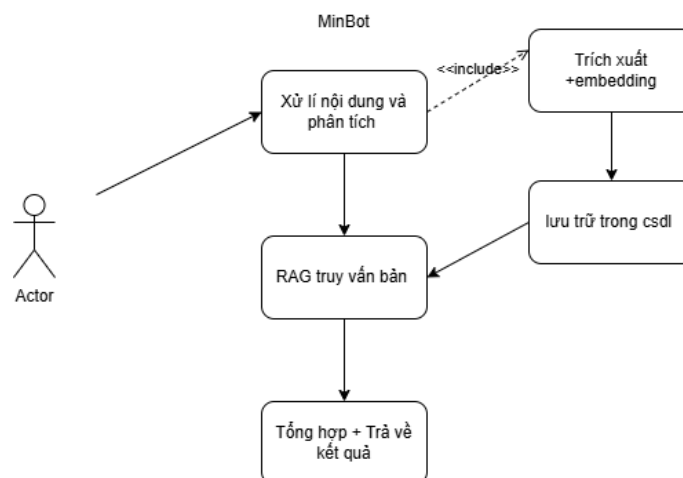


Figure 3.4: Sơ đồ Usecase Truy xuất dữ liệu (RAG)

3.2.2.5 UseCase: Tìm kiếm thông tin Web

Mô tả chức năng: Khi hệ thống không thể trả lời bằng thông tin có sẵn, người dùng có thể yêu cầu MinBot tìm kiếm trực tuyến để mở rộng phạm vi dữ liệu.

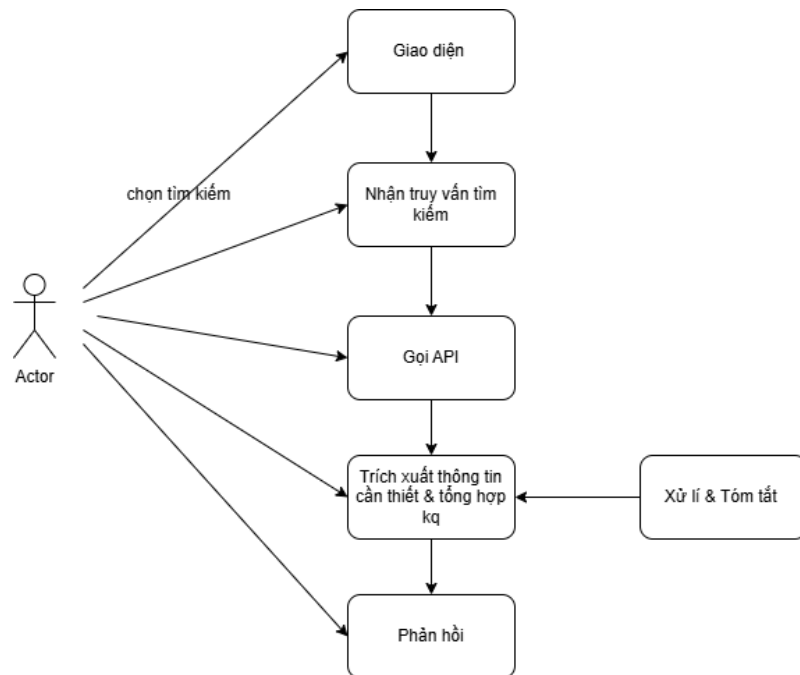


Figure 3.5: Sơ đồ Usecase Tìm kiếm thông tin từ Web

3.2.2.6 Usecase: Xem lại lịch sử hội thoại

Mô tả chức năng: Người dùng có thể xem lại các câu hỏi và câu trả lời trước đó thông qua mục "Lịch sử hội thoại".

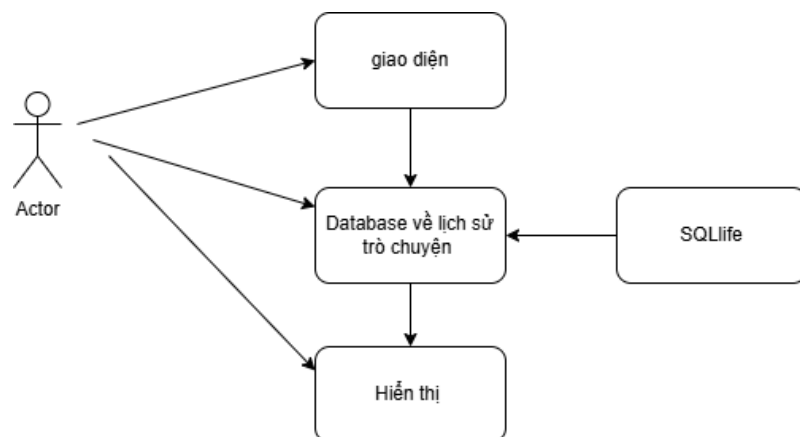


Figure 3.6: Sơ đồ Usecase xem lịch sử hội thoại

3.3 Thiết kế Luồng dữ liệu (Data Flow Diagram - DFD)

3.3.1 Mô tả DFD – Sơ đồ luồng dữ liệu tổng quát

DFD mô tả cách dữ liệu di chuyển trong hệ thống. Chúng ta có hai luồng chính:

- **Luồng 1: Truy vấn từ người dùng (User Query Flow)**
Người dùng truy cập vào giao diện và gửi truy vấn, backend sẽ xử lý logic truy vấn đó và sẽ đưa ra lựa chọn giữa việc gửi truy vấn trực tiếp đến API hoặc RAG để truy xuất thông tin kho dữ liệu. Kết quả sẽ được trả về dựa trên truy vấn và hiện thị ra màn hình (giao diện)
- **Luồng 2: Tải lên và xử lý tài liệu (Document Upload Flow)**
Người dùng tải (đính kèm) tài liệu, tệp văn bản lên MinBot, file sẽ được backend xử lý và nội dung sẽ được embedding và lưu vào cơ sở dữ liệu để phục vụ cho các lần truy vấn sau - RAG

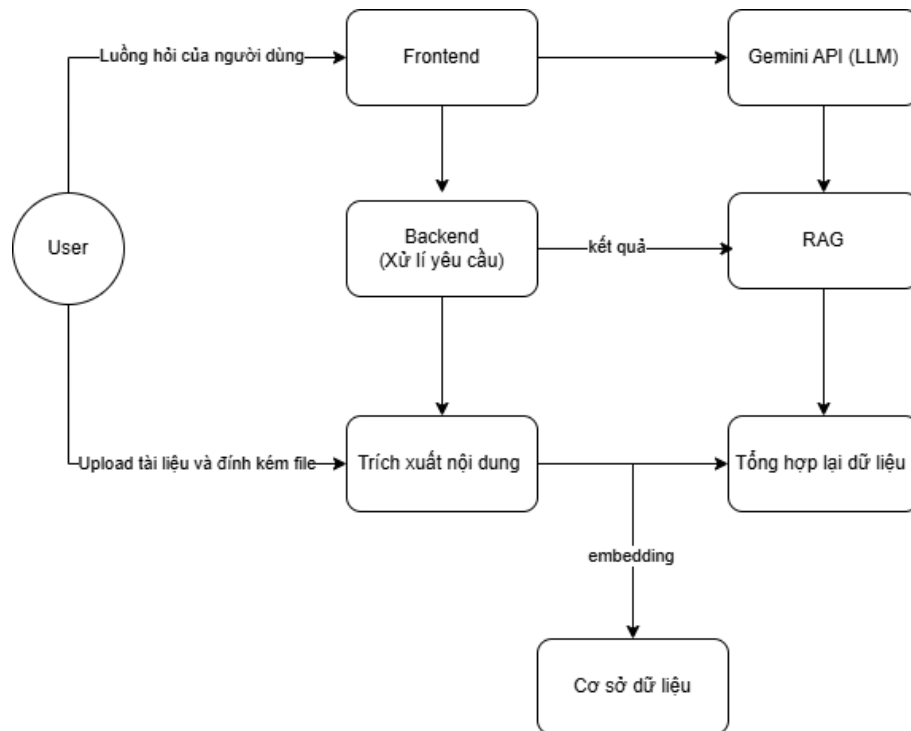


Figure 3.7: DFD chính của MinBot

3.3.2 Các thực thể chính trong DFD

Thực thể	Vai trò
User	Người dùng hệ thống, thực hiện truy vấn hoặc tải tài liệu.
Frontend	Giao diện hiển thị và tiếp nhận đầu vào, phản hồi từ người dùng.
Backend	Xử lý logic, định tuyến yêu cầu đến các dịch vụ thích hợp.
Gemini API	Mô hình ngôn ngữ lớn để sinh câu trả lời từ truy vấn đầu vào.
RAG Module	Truy xuất các đoạn văn bản liên quan từ vector database.
Vector DB	Kho lưu trữ embedding từ tài liệu người dùng tải lên.

Table 3.5: Bảng mô tả vai trò của các thực thể trong hệ thống

CHƯƠNG 4 Kiến trúc Hệ thống

Chương này trình bày chi tiết về kiến trúc của MinBot, một hệ thống chatbot thông minh xây dựng theo mô hình RAG (Retrieval-Augmented Generation). Hệ thống bao gồm backend **FastAPI**, frontend **React**, và sử dụng **FAISS** để lập chỉ mục vector. Toàn bộ kiến trúc được thiết kế theo mô-đun và container hóa bằng Docker để dễ dàng triển khai và bảo trì.

4.1 Kiến trúc tổng thể

MinBot được thiết kế theo mô hình client-server, gồm hai thành phần chính:

- **Frontend:** Một ứng dụng trang đơn (Single Page Application) xây dựng bằng React, cung cấp giao diện người dùng để tương tác với chatbot.
- **Backend:** Một API server phát triển bằng FastAPI, chịu trách nhiệm xử lý logic nghiệp vụ, quản lý dữ liệu và tích hợp với các dịch vụ bên ngoài.

Hai dịch vụ này giao tiếp với nhau qua một mạng nội bộ được quản lý bởi Docker Compose. Hình 4.1 minh họa kiến trúc tổng quan của hệ thống.

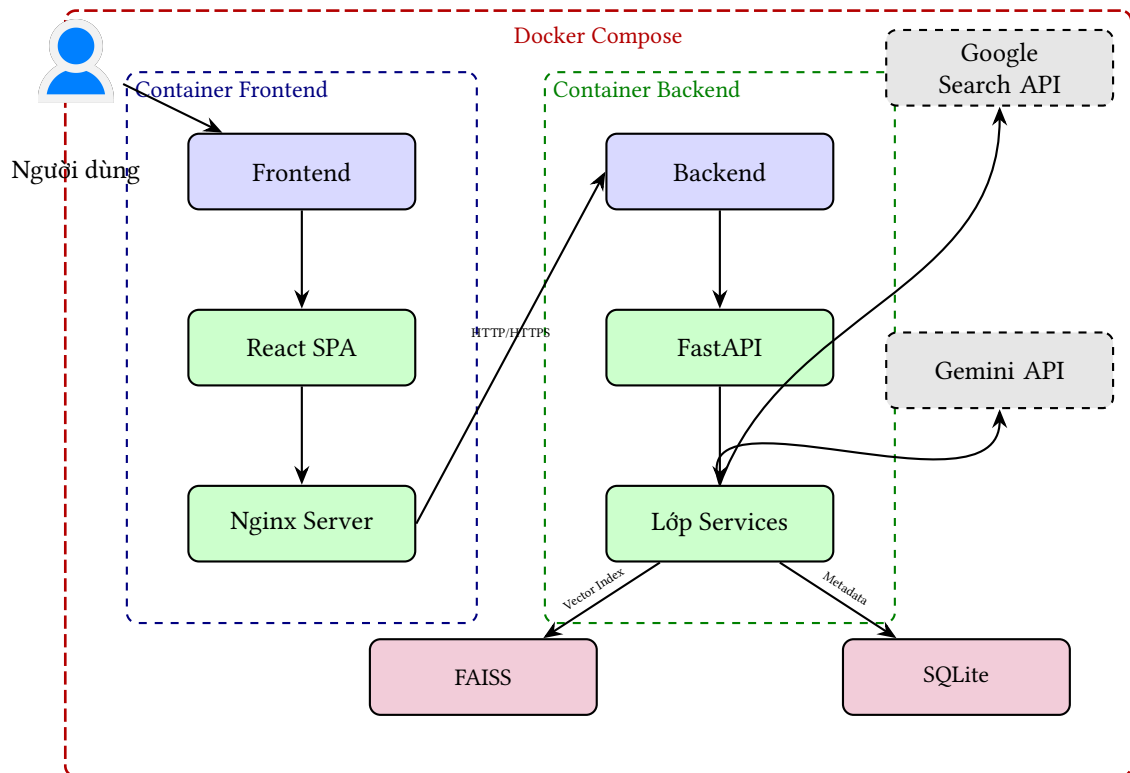


Figure 4.1: Kiến trúc tổng thể của hệ thống MinBot.

4.2 Kiến trúc Backend

Backend được thiết kế theo kiến trúc phân lớp (Layered Architecture) nhằm tách biệt các mối quan tâm, giúp hệ thống dễ dàng bảo trì và mở rộng.

4.2.1 Cấu trúc phân lớp

Hệ thống Backend bao gồm 3 lớp chính (Hình 4.2):

1. **API Layer (/api):** Lớp giao tiếp ngoài cùng, định nghĩa các RESTful endpoint. Lớp này sử dụng Pydantic schema để xác thực (validation) và tuần tự hoá (serialization) dữ liệu.
2. **Service Layer (/services):** Chứa toàn bộ logic nghiệp vụ của ứng dụng như xử lý tệp, RAG, quản lý hội thoại và tương tác với các dịch vụ ngoài.
3. **Persistence Layer:** Chịu trách nhiệm lưu trữ và truy xuất dữ liệu, bao gồm:
 - **SQLite:** Cơ sở dữ liệu quan hệ để lưu trữ siêu dữ liệu (metadata) của hội thoại, tin nhắn và các tệp đã tải lên.
 - **FAISS (Facebook AI Similarity Search):** Thư viện hiệu năng cao để lập chỉ mục và tìm kiếm tương tự trên các vector embedding, là lõi của chức năng RAG.

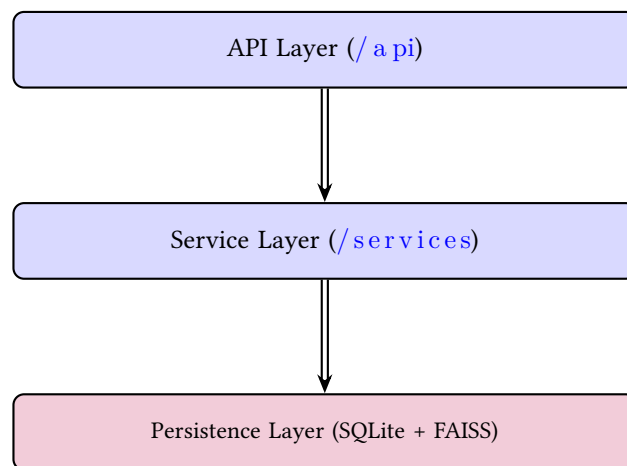


Figure 4.2: Các lớp kiến trúc của Backend.

4.2.2 Thiết kế Cơ sở dữ liệu

Hệ thống sử dụng SQLite để lưu trữ siêu dữ liệu về các cuộc hội thoại và tin nhắn. Cấu trúc của các bảng chính được mô tả trong Bảng 4.1 và Bảng 4.2.

Cột	Kiểu	Mô tả
id	INTEGER (PK)	Khoá chính, tự tăng
user_id	INTEGER	ID người dùng sở hữu cuộc hội thoại
title	TEXT	Tiêu đề cuộc hội thoại
created_at	DATETIME	Thời gian tạo
updated_at	DATETIME	Thời gian cập nhật cuối

Table 4.1: Schema bảng `conversations` trong SQLite.

Cột	Kiểu	Mô tả
id	INTEGER (PK)	Khoá chính, tự tăng
conversation_id	INTEGER (FK)	Tham chiếu tới bảng <code>conversations</code>
role	TEXT	Vai trò (user / assistant)
content	TEXT	Nội dung tin nhắn
created_at	DATETIME	Thời gian tạo

Table 4.2: Schema bảng `messages` trong SQLite.

4.2.3 Các thành phần dịch vụ

Lớp Service được chia thành các module nhỏ, mỗi module đảm nhiệm một chức năng cụ thể và tương tác với nhau để hoàn thành một yêu cầu phức tạp (Hình 4.3).

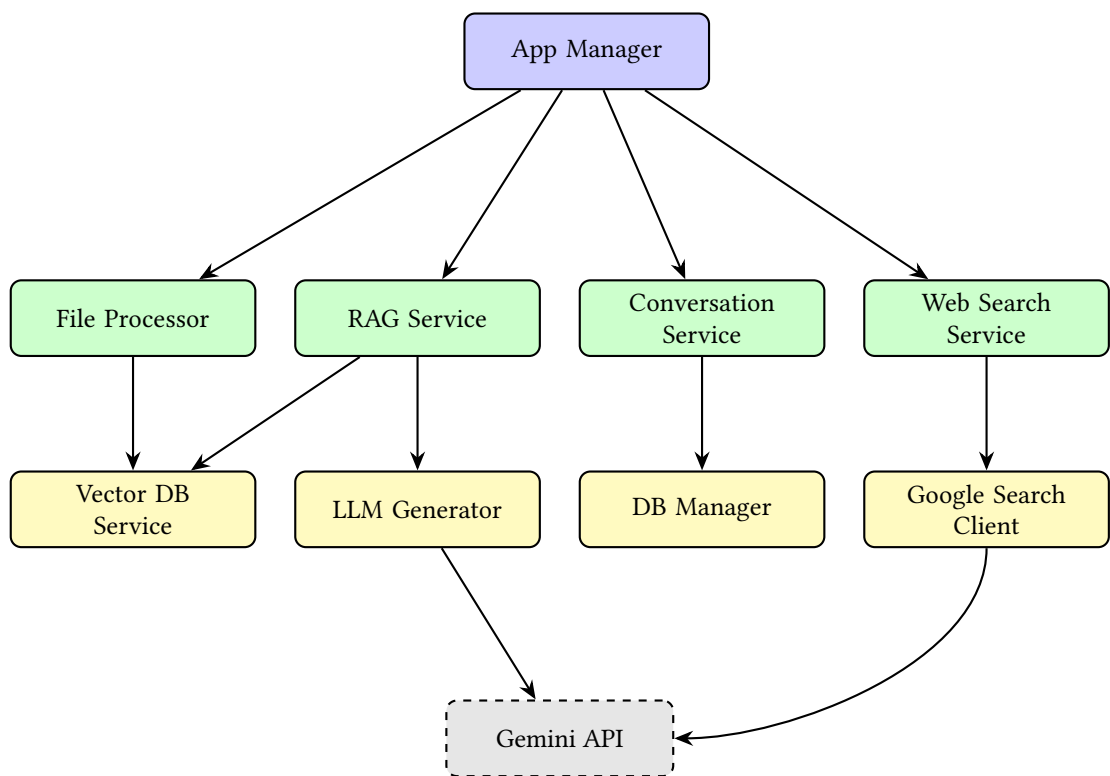


Figure 4.3: Sơ đồ tương tác giữa các thành phần trong lớp Service của Backend.

4.2.4 Các luồng xử lý chính

4.2.4.1 Pipeline xử lý tệp tin

Khi người dùng tải lên một tệp, hệ thống thực hiện một chuỗi xử lý bất đồng bộ để trích xuất, xử lý và lưu trữ nội dung cho RAG (Hình 4.4).

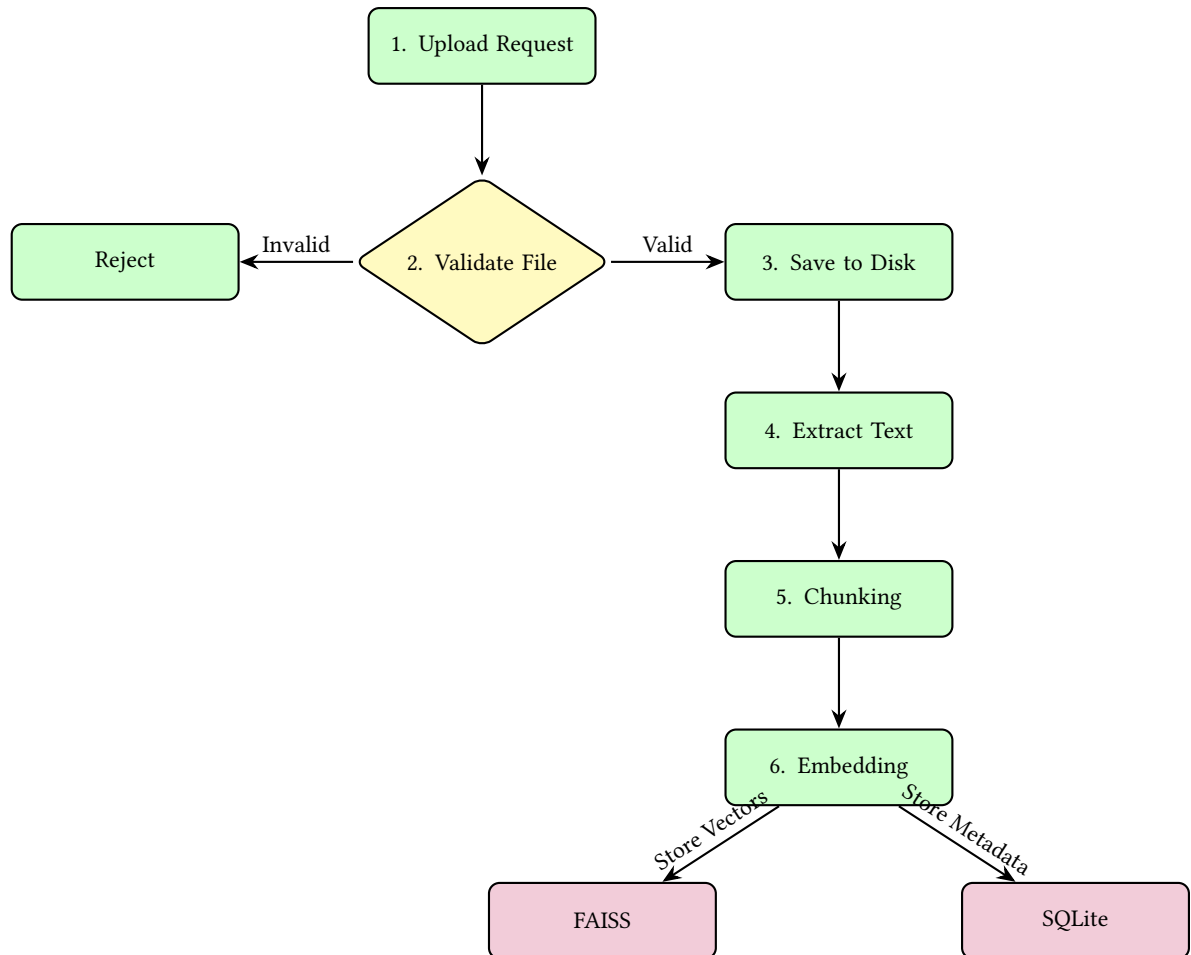


Figure 4.4: Pipeline xử lý tệp tin đầu vào.

4.2.4.2 Pipeline RAG

Khi nhận một câu hỏi, pipeline RAG được kích hoạt để tìm kiếm thông tin liên quan từ các tệp đã xử lý và tạo ra câu trả lời (Hình 4.5).

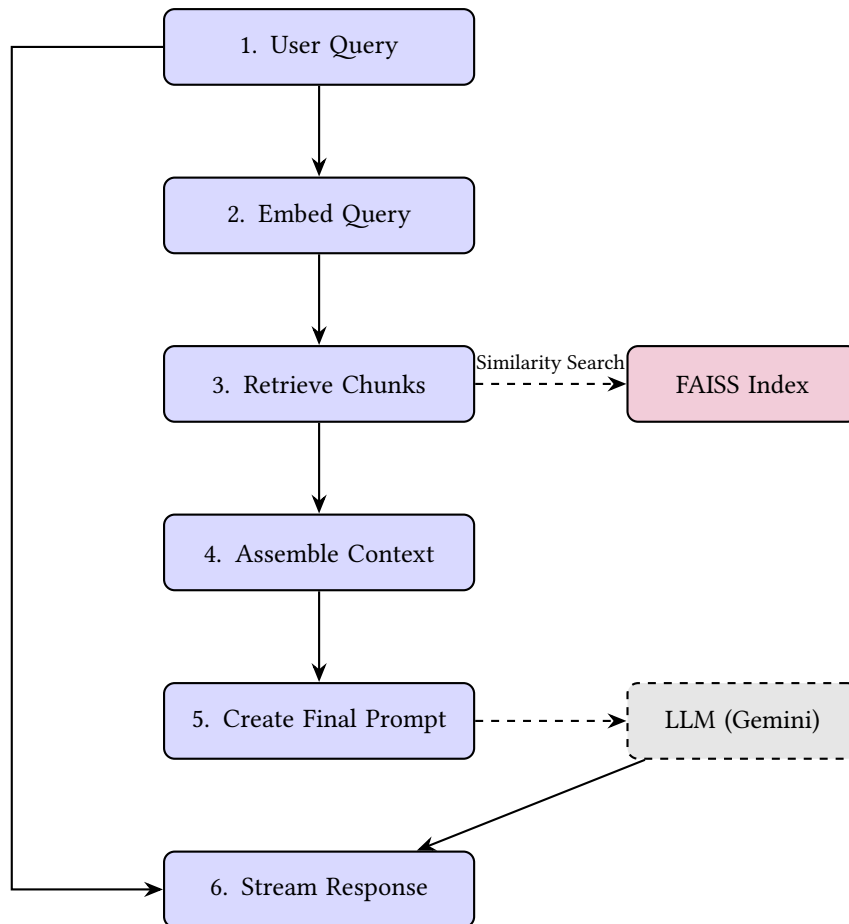


Figure 4.5: Pipeline sinh truy vấn tăng cường (RAG) trong MinBot.

4.3 Kiến trúc Frontend

Frontend là một ứng dụng SPA được phát triển bằng **React** và **TypeScript**, sử dụng **Vite** làm công cụ build. Giao diện người dùng được xây dựng từ các component có thể tái sử dụng.

4.3.1 Cấu trúc Component

Kiến trúc component được tổ chức theo chức năng, với các component cha quản lý trạng thái và luồng dữ liệu cho các component con (Hình 4.6).

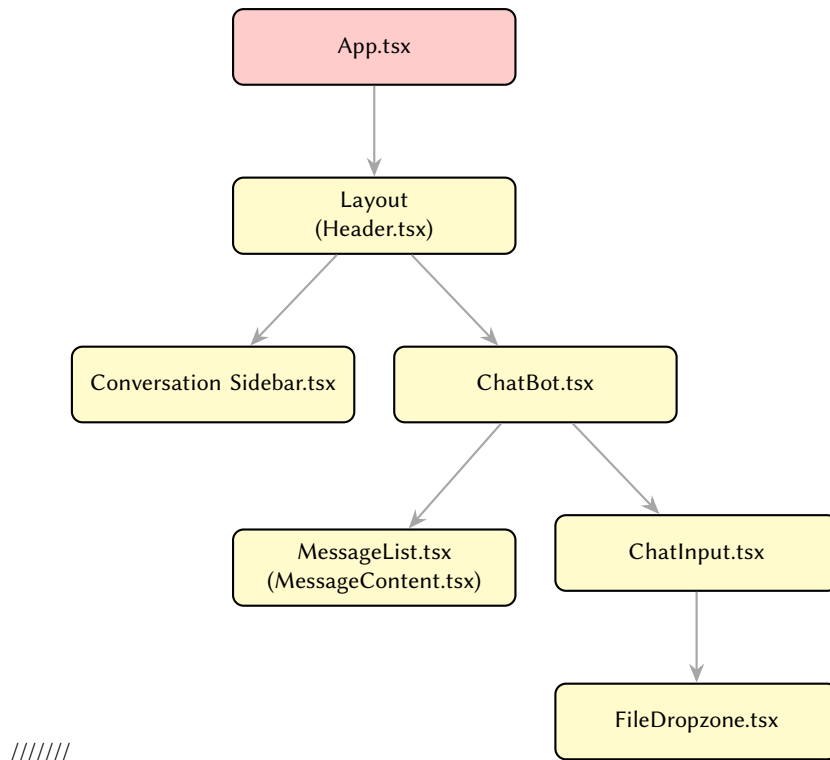


Figure 4.6: Sơ đồ phân cấp các thành phần chính của Frontend.

4.3.2 Quản lý trạng thái và Tương tác API

- **Quản lý trạng thái:** Trạng thái ứng dụng (danh sách hội thoại, tin nhắn,...) được quản lý bằng React Hooks (`'useState'`, `'useEffect'`).
- **Tương tác API:** Thư viện **Axios** được sử dụng để gửi các yêu cầu HTTP đến Backend. Một lớp service client ([/services/api.ts](#)) được tạo ra để quản lý tập trung các hàm gọi API.

4.4 Triển khai và Vận hành

4.4.1 Containerization với Docker

Hệ thống được đóng gói và triển khai bằng Docker và Docker Compose.

- **Dockerfile.backend:** Xây dựng image cho ứng dụng FastAPI trên nền Python 3.11, cài đặt thư viện từ [requirements.txt](#) và khởi chạy bằng Uvicorn.
- **Dockerfile.frontend:** Sử dụng multi-stage build. Giai đoạn 1 dùng Node.js để build ứng dụng React, giai đoạn 2 sao chép các tệp tĩnh vào image Nginx để phục vụ người dùng.
- **docker-compose.yml:** Định nghĩa và liên kết các service [frontend](#) và [backend](#), cấu hình mạng và volume chia sẻ dữ liệu.

4.4.2 Logging và Giám sát

Hệ thống tích hợp cơ chế ghi log để theo dõi hoạt động và hỗ trợ gỡ lỗi.

- Logger trung tâm được cấu hình trong `main.py` của Backend, ghi log ra console và file.
- Mỗi request được gán một ID duy nhất để tiện cho việc truy vết (tracing).
- Các lỗi nghiệp vụ và hệ thống được bắt và xử lý tường minh, trả về mã trạng thái HTTP phù hợp cùng thông báo lỗi rõ ràng.

CHƯƠNG 5 THỰC THI HỆ THỐNG

Chương này trình bày chi tiết quá trình và các quyết định kỹ thuật trong việc xây dựng hệ thống MinBot. Nội dung tập trung vào bốn khía cạnh chính: kiến trúc hệ thống tổng thể, pipeline RAG lõi, hiện thực chi tiết backend và frontend, cùng với các giải pháp về triển khai, bảo mật và tối ưu hóa hiệu năng.

5.1 Kiến trúc tổng thể

Kiến trúc của MinBot được thiết kế theo mô hình microservices, phân tách rõ ràng giữa giao diện người dùng, lớp API gateway và các dịch vụ nghiệp vụ backend. Mô hình này không chỉ giúp tăng tính module hóa, dễ bảo trì mà còn cho phép các thành phần được phát triển và mở rộng độc lập.

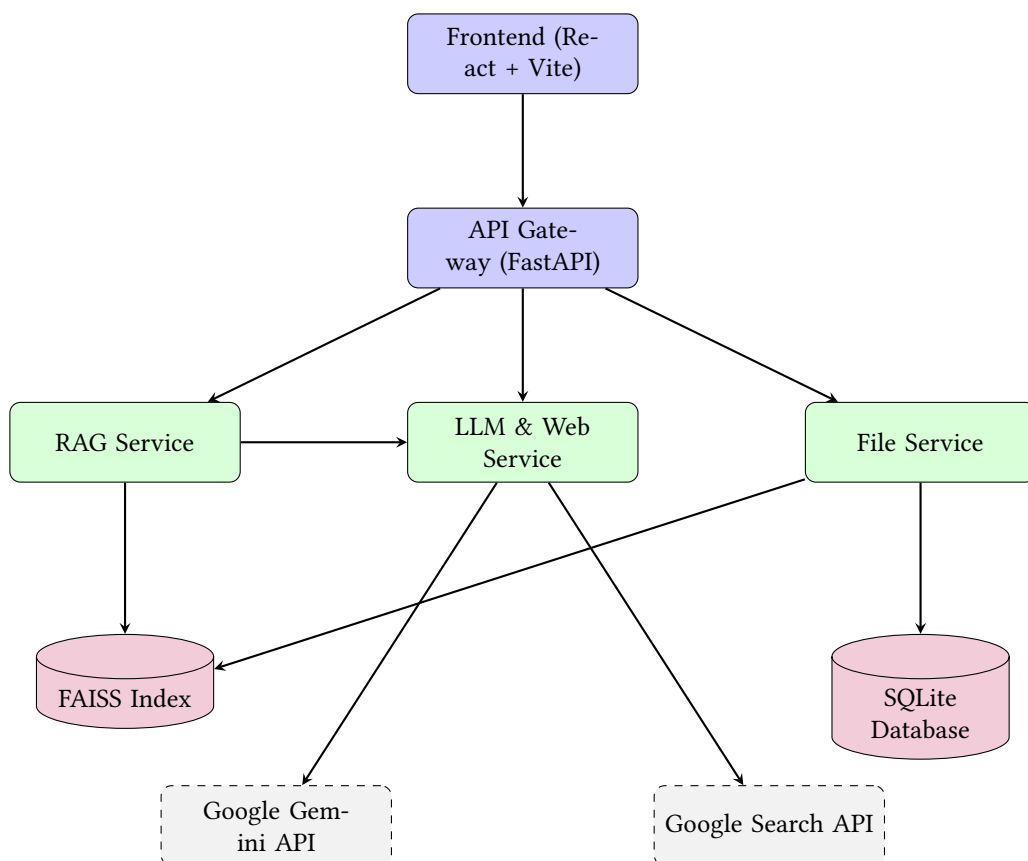


Figure 5.1: Kiến trúc triển khai tổng thể của MinBot

Các thành phần chính bao gồm:

- **Frontend:** Là một Single Page Application (SPA) xây dựng bằng React và Vite, chịu trách nhiệm cho toàn bộ giao diện và trải nghiệm người dùng.

- **API Gateway:** Một cổng FastAPI đóng vai trò là điểm vào duy nhất cho mọi yêu cầu từ frontend, điều phối chúng đến các service nghiệp vụ tương ứng.
- **Backend Services:** Các logic nghiệp vụ được chia thành ba services riêng biệt:
 - *RAG Service:* Thực thi logic truy xuất thông tin từ tài liệu đã được vector hóa.
 - *File Service:* Quản lý vòng đời của tệp (tải lên, xử lý, lưu trữ siêu dữ liệu).
 - *LLM & Web Service:* Đóng gói các tương tác với API bên ngoài (Google Gemini, Google Search) và logic sinh ngôn ngữ.
- **Data Stores:**
 - *FAISS Index:* Lưu trữ và phục vụ truy vấn vector tương đồng cho tính năng RAG.
 - *SQLite:* Lưu trữ siêu dữ liệu của tệp, lịch sử hội thoại và các thông tin quan hệ khác.

5.2 Pipeline xử lý lỗi: Retrieval-Augmented Generation

5.2.1 Luồng xử lý truy vấn RAG

Pipeline RAG là trái tim của hệ thống, được kích hoạt mỗi khi người dùng gửi một truy vấn. Điểm đặc biệt trong thiết kế của MinBot là khả năng truy xuất song song từ hai nguồn tri thức: tri thức nội bộ từ tài liệu người dùng và tri thức thời gian thực từ Internet.

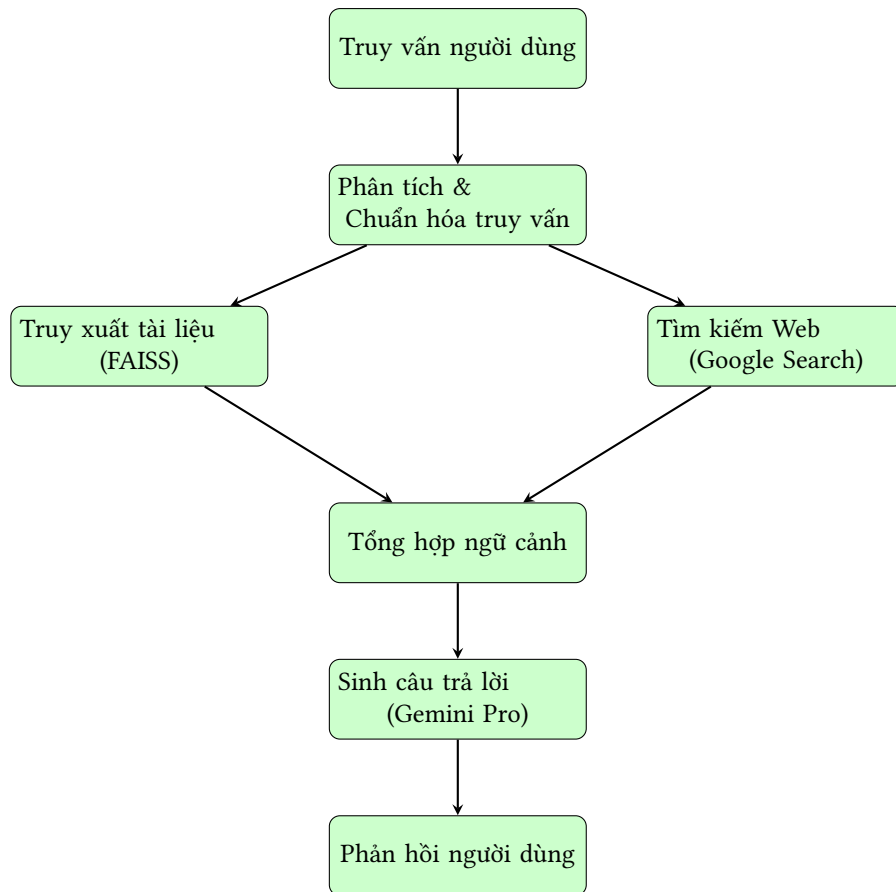


Figure 5.2: Pipeline RAG song song truy xuất tri thức nội bộ và web

5.2.2 Luồng xử lý và Vector hóa tài liệu

Để có thể truy xuất, tài liệu PDF do người dùng tải lên phải trải qua một quy trình xử lý và vector hóa được thực hiện bởi [FileService](#).

1. **Trích xuất văn bản:** Sử dụng thư viện PyMuPDF để trích xuất nội dung text từ tệp PDF.
2. **Tiền xử lý:** Làm sạch văn bản, loại bỏ các ký tự thừa, định dạng lại các khoảng trắng.
3. **Phân đoạn (Chunking):** Văn bản được chia thành các đoạn nhỏ (chunks) với kích thước và độ chồng lấp (overlap) được tính toán hợp lý để đảm bảo ngữ nghĩa không bị phá vỡ.
4. **Vector hóa (Embedding):** Mỗi chunk được chuyển đổi thành một vector số thực bằng mô hình embedding của Google.
5. **Lưu trữ:** Các vector được lưu vào FAISS Index. Đồng thời, một bản ghi ảnh xạ giữa ID của chunk, nội dung và tệp gốc được tạo trong SQLite.

5.3 Môi trường triển khai (Docker Compose)

Để đơn giản hóa quá trình cài đặt và đảm bảo tính nhất quán giữa các môi trường (phát triển, kiểm thử, sản phẩm), dự án sử dụng Docker và Docker Compose.

- **Dockerfile:** Mỗi service (backend, frontend) có một [Dockerfile](#) riêng, định nghĩa các bước để xây dựng một image chứa tất cả các dependencies và mã nguồn cần thiết.
- **Docker Compose:** File [docker-compose.yml](#) định nghĩa và điều phối các container. Nó tự động hóa việc khởi tạo network, liên kết các service và mount volumes để lưu trữ dữ liệu bền vững (persistent storage) cho FAISS index và SQLite database.

Giải pháp này cho phép toàn bộ hệ thống có thể được khởi chạy chỉ bằng một lệnh duy nhất ([docker-composeup](#)), loại bỏ các xung đột về thư viện và môi trường.

5.4 Bảo mật và Giám sát

5.4.1 Bảo mật

Các biện pháp bảo mật cơ bản đã được tích hợp vào hệ thống:

- **Quản lý API Key:** Các API key cho dịch vụ của Google được quản lý thông qua biến môi trường và nạp vào ứng dụng một cách an toàn, không bị hard-code trong mã nguồn. Hệ thống cũng cung cấp giao diện để người dùng tự nhập API key của họ.
- **Xác thực CORS:** Backend FastAPI được cấu hình chính sách Cross-Origin Resource Sharing (CORS) nghiêm ngặt, chỉ cho phép các yêu cầu từ tên miền của frontend, ngăn chặn các trang web độc hại gọi đến API.
- **Kiểm tra tệp tải lên:** Hệ thống kiểm tra định dạng và kích thước tệp tải lên để đảm bảo chỉ các tệp PDF hợp lệ được xử lý, giảm thiểu nguy cơ từ các tệp độc hại.

5.4.2 Giám sát

- **Logging:** Hệ thống sử dụng thư viện logging tiêu chuẩn của Python và FastAPI để ghi lại các sự kiện quan trọng, bao gồm các yêu cầu API, lỗi xử lý và các cảnh báo hệ thống. Log được định dạng JSON để dễ dàng tích hợp với các hệ thống thu thập log tập trung như ELK Stack hoặc Splunk.
- **Health Check:** Một endpoint [/health](#) được cung cấp để các hệ thống giám sát bên ngoài (ví dụ: Uptime Kuma, Prometheus) có thể kiểm tra trạng thái hoạt động của ứng dụng.

5.5 Tối ưu hóa và Xử lý Bất đồng bộ

Hiệu năng là một yếu tố quan trọng trong trải nghiệm người dùng. MinBot áp dụng các kỹ thuật sau để tối ưu hóa tốc độ phản hồi:

- **Xử lý bất đồng bộ:** Tận dụng [asyncio](#) của Python và khả năng của FastAPI, tất cả các tác vụ I/O-bound (gọi API ngoài, truy vấn database, đọc/ghi file) đều được thực thi bất đồng bộ. Điều này cho phép server xử lý hàng ngàn kết nối đồng thời mà không bị chặn.
- **Batch Processing:** Trong quá trình vector hóa tài liệu, thay vì xử lý từng chunk một, hệ thống gom chúng thành các lô (batches) và gửi đến mô hình embedding trong một lần gọi API duy nhất. Kỹ thuật này giảm đáng kể tổng thời gian cần thiết để xử lý một tài liệu lớn.
- **Error Recovery:** Hệ thống có cơ chế tự động thử lại (retry) với khoảng thời gian tăng dần (exponential backoff) khi các cuộc gọi API bên ngoài thất bại. Trong trường hợp RAG gặp lỗi nghiêm trọng, hệ thống sẽ tự động fallback về chế độ chatbot thông thường (chỉ dùng LLM) để đảm bảo không làm gián đoạn cuộc hội thoại.

CHƯƠNG 6 KIỂM THỬ

Kiểm thử phần mềm là một quy trình không thể thiếu trong vòng đời phát triển, giữ vai trò then chốt trong việc đảm bảo chất lượng sản phẩm. Giai đoạn này không chỉ nhằm mục đích xác minh các yêu cầu chức năng đã được đáp ứng, mà còn để đánh giá độ tin cậy, hiệu năng và tính bảo mật của hệ thống. Chương này trình bày một cách có hệ thống về chiến lược kiểm thử, các kịch bản kiểm thử chi tiết được thiết kế cho dự án Chatbot, cùng với kết quả thực nghiệm và phân tích sâu sắc các lỗi được phát hiện.

6.1 Kế hoạch Kiểm thử

6.1.1 Mục tiêu kiểm thử

- **Xác minh yêu cầu chức năng:** Đảm bảo tất cả các chức năng đã định nghĩa (tải file, chat, quản lý hội thoại, v.v.) hoạt động đúng như đặc tả.
- **Kiểm tra tính đúng đắn của luồng RAG:** Xác thực rằng Chatbot có khả năng truy xuất đúng thông tin từ tài liệu được cung cấp và sử dụng thông tin đó để tạo ra câu trả lời phù hợp, có ngữ cảnh.
- **Đảm bảo trải nghiệm người dùng (UX):** Kiểm tra giao diện người dùng có trực quan, dễ sử dụng, không có lỗi hiển thị và các tương tác mượt mà.
- **Xác minh tính ổn định:** Đảm bảo hệ thống không bị sập ([crash](#)) khi xử lý các trường hợp ngoại lệ hoặc dữ liệu đầu vào không hợp lệ.
- **Đánh giá hiệu năng cơ bản:** Kiểm tra thời gian phản hồi của hệ thống đối với các truy vấn thông thường và khi đang tạo câu trả lời.

6.1.2 Phạm vi và Phương pháp kiểm thử

Dựa trên đặc thù của dự án, một chiến lược kiểm thử đa tầng đã được áp dụng, kết hợp cả kiểm thử tự động và thủ công.

Các cấp độ và loại hình kiểm thử

- **Kiểm thử API (API Testing):** Sử dụng bộ kiểm thử tự động Pytest để xác minh tính đúng đắn của từng endpoint trong Backend, đảm bảo chúng tuân thủ đặc tả và xử lý các trường hợp biên.
- **Kiểm thử Tích hợp (Integration Testing):** Tập trung vào sự tương tác giữa Frontend và Backend, đảm bảo các lời gọi API từ giao diện được xử lý chính xác.

- **Kiểm thử Giao diện Người dùng (UI/E2E Testing):** Sử dụng Playwright để tự động hóa các kịch bản sử dụng thực tế trên trình duyệt, kiểm tra luồng hoạt động end-to-end từ góc nhìn người dùng.
- **Kiểm thử Thủ công và Phân tích Mã nguồn (Manual & White-box Testing):** Bổ sung cho kiểm thử tự động, phương pháp này tập trung vào việc phân tích logic nghiệp vụ phức tạp và các luồng hoạt động chưa được tự động hóa.

Môi trường kiểm thử (Test Environment)

- **Frontend (Tự động):** Playwright trên môi trường ảo (headless browser).
- **Frontend (Thủ công):** Google Chrome 12x, Mozilla Firefox 12x trên Windows 11.
- **Backend (Tự động):** Pytest với môi trường Python 3.10.
- **Backend (Phân tích mã nguồn):** Môi trường phát triển Visual Studio Code.

6.2 Thiết kế Ca kiểm thử (Test Cases)

Các ca kiểm thử được xây dựng để bao phủ các kịch bản từ cơ bản đến các trường hợp biên, đặc biệt chú trọng vào việc xác thực tính chính xác của lõi RAG và các cổng vào của hệ thống.

6.2.1 Chức năng: Tải và Xử lý Tài liệu

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-F-01	Tải lên file .pdf hợp lệ	1. Nhấn nút "Tài tài liệu". 2. Chọn một file .pdf hợp lệ.	Hệ thống báo "Xử lý thành công". File được thêm vào cơ sở tri thức.
TC-F-02	Tải lên file có định dạng không hỗ trợ	1. Nhấn nút "Tài tài liệu". 2. Chọn một file .zip hoặc .docx.	Hệ thống báo lỗi "Định dạng file không được hỗ trợ".
TC-F-03	Tải lên file .pdf bị hỏng	1. Nhấn nút "Tài tài liệu". 2. Chọn một file .pdf bị hỏng.	Hệ thống báo lỗi "Không thể xử lý file này".
TC-F-04	Tải lên file có tên tiếng Việt, ký tự đặc biệt	1. Đổi tên 1 file .pdf thành "Tài liệu (2024).pdf". 2. Thực hiện tải lên.	File được tải lên và xử lý thành công, tên file hiển thị đúng.
TC-F-05	Tải file vượt quá dung lượng cho phép	1. Tải lên một file 30MB (giả sử giới hạn là 25MB).	Hệ thống từ chối và báo "Dung lượng file quá lớn".
TC-F-06	Xác thực MIME type (chặn file nguy hiểm)	1. Đổi tên file malware.exe thành malware.pdf. 2. Tải lên file đã đổi tên.	Hệ thống từ chối và báo lỗi "Loại file không hợp lệ" sau khi kiểm tra MIME type.
TC-F-07	Thiếu API Key khi tải file	1. Gửi request POST /files/upload không kèm header x-api-key.	Hệ thống trả về HTTP 401 Unauthorized.

Table 6.1: Test cases cho chức năng Tải và Xử lý Tài liệu

6.2.2 Chức năng: Trò chuyện với Chatbot (Lỗi RAG)

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-C-01	RAG với thông tin không phổ biến	1. Tải file: "Dự án MinBot có mã hiệu XZ-7." 2. Hỏi: "Mã hiệu dự án MinBot là gì?".	Chatbot trả lời chính xác "XZ-7".
TC-C-02	Hỏi câu không có trong tài liệu	1. Sử dụng tài liệu ở TC-C-01. 2. Hỏi: "Ngân sách dự án?".	Chatbot trả lời không tìm thấy thông tin.
TC-C-03	Gửi tin nhắn rỗng	1. Không nhập gì, nhấn "Gửi".	Nút gửi bị vô hiệu hóa, không có tin nhắn được gửi.
TC-C-04	Hội thoại nhiều lượt (ghi nhớ ngữ cảnh)	1. Hỏi: "Ai quản lý dự án Min-Bot?". 2. Sau khi có câu trả lời, hỏi: "Thông tin liên hệ của người đó?".	Chatbot hiểu "người đó" và trả lời đúng.
TC-C-05	Dừng tạo câu trả lời	1. Hỏi một câu phức tạp. 2. Trong lúc Chatbot đang tạo câu trả lời, nhấn nút "Dừng".	Luồng tạo câu trả lời dừng lại ngay lập tức.
TC-C-06	RAG tổng hợp thông tin	1. Tải file: "Phòng A có 5 người. Phòng B có 7 người." 2. Hỏi: "Tổng số người của hai phòng A và B?".	Chatbot trả lời "12 người".
TC-C-07	Truy vấn rỗng tới API	1. Gửi POST /rag/query với payload { "question": "" }.	Backend trả về HTTP 422 Unprocessable Entity với thông báo lỗi rõ ràng.
TC-C-08	<code>conversation_id</code> không tồn tại	1. Gửi POST /rag/query với <code>conversation_id</code> = 9999.	Backend trả về HTTP 404 Not Found.

Table 6.2: Test cases cho chức năng Trò chuyện

6.2.3 Chức năng: Quản lý Hội thoại

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-M-01	Tạo hội thoại mới	1. Đang ở một hội thoại. 2. Nhấn nút "Hội thoại mới".	Giao diện chat được làm mới. Một mục hội thoại mới được tạo và chọn.
TC-M-02	Chuyển đổi hội thoại	1. Có ít nhất 2 hội thoại. 2. Nhấn vào một hội thoại cũ hơn.	Nội dung chat của hội thoại cũ được tải lại chính xác.
TC-M-03	Xóa hội thoại	1. Di chuột qua hội thoại. 2. Nhấn nút xóa và xác nhận.	Hội thoại biến mất khỏi danh sách. Giao diện chuyển về hội thoại khác.
TC-M-04	Đổi tên hội thoại	1. Gửi POST /conversation/re-name với tên mới.	Tên hội thoại được cập nhật trong DB và phản hồi chứa tên mới.
TC-M-05	Xóa hội thoại không tồn tại	1. Gửi DELETE /conversation/123456.	Backend trả về HTTP 404 Not Found.
TC-M-06	Lấy danh sách hội thoại của user chưa có hội thoại	1. Gọi GET /conversation/user/{new_user_id}.	API trả về mảng rỗng và HTTP 200.

Table 6.3: Test cases cho chức năng Quản lý Hội thoại

6.2.4 Giao diện: Tải và Xử lý Tài liệu (Frontend)

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-UI-F-01	Kéo-thả file PDF hợp lệ	1. Kéo file sample.pdf vào vùng dropzone.	Progress bar hiển thị, hoàn tất với thông báo "Xử lý thành công".
TC-UI-F-02	Thông báo lỗi định dạng không hỗ trợ	1. Kéo file malware.exe .	Hiển thị toast "Định dạng file không được hỗ trợ".
TC-UI-F-03	Hiển thị tiến trình upload chính xác	1. Tải file 10MB.	Progress tăng dần từ 0% đến 100%.
TC-UI-F-04	Cảnh báo dung lượng vượt giới hạn	1. Kéo file 30MB.	Hiển thị toast "Dung lượng file quá lớn".
TC-UI-F-05	Upload nhiều file liên tiếp	1. Chọn 3 file PDF và tải lên tuần tự.	Mỗi file hiển thị progress riêng, UI không chông chéo.

Table 6.4: Test cases giao diện Tải và Xử lý Tài liệu

6.2.5 Giao diện: Trò chuyện với Chatbot

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-UI-C-01	Vô hiệu hoá nút gửi khi ô chat rỗng	1. Để trống ô nhập, quan sát nút "Gửi".	Nút "Gửi" bị disable.
TC-UI-C-02	Hiển thị tin nhắn streaming	1. Gửi câu hỏi dài.	Tin nhắn của chatbot hiển thị từng ký tự theo thời gian thực.
TC-UI-C-03	Nút "Dừng" hoạt động	1. Trong khi chatbot đang trả lời, nhấn "Dừng".	Tin nhắn dừng ngay, nút "Dừng" ẩn.
TC-UI-C-04	Render Markdown	1. Chatbot trả về đoạn code.	UI hiển thị block code được highlight đúng.
TC-UI-C-05	Hiển thị lỗi backend	1. Gửi truy vấn gây lỗi 500.	UI hiện toast "Có lỗi xảy ra" và không crash.

Table 6.5: Test cases giao diện Trò chuyện

6.2.6 Giao diện: Quản lý Hội thoại

ID	Mô tả	Các bước thực hiện	Kết quả mong đợi
TC-UI-M-01	Tạo hội thoại mới	1. Nhấn biểu tượng "+" trong sidebar.	Thẻ hội thoại mới xuất hiện và được chọn.
TC-UI-M-02	Đổi tên hội thoại	1. Nhấn đúp tên hội thoại. 2. Nhập tên mới và Enter.	Tên được cập nhật ngay.
TC-UI-M-03	Xóa hội thoại	1. Hover và nhấn biểu tượng thùng rác.	Thẻ biến mất, UI ổn định.
TC-UI-M-04	Scroll danh sách hội thoại dài	1. Tạo >20 hội thoại.	Thanh cuộn xuất hiện, scroll mượt.
TC-UI-M-05	Highlight hội thoại đang chọn	1. Chọn một hội thoại.	Thẻ được chọn có nền khác, rõ ràng.

Table 6.6: Test cases giao diện Quản lý Hội thoại

6.3 Kết quả và Phân tích

6.3.1 Bảng tổng hợp kết quả kiểm thử

Dựa trên phân tích mã nguồn và thực thi thủ công, kết quả kiểm thử được tổng hợp như sau:

ID	Kết quả thực tế	Trạng thái	Ghi chú
TC-F-01	File được lưu thành công vào hệ thống.	Pass	
TC-F-02	Hệ thống chấp nhận mọi định dạng file, có thể gây lỗi ở bước xử lý sau.	Fail	Bug #F-1
TC-F-03	Hệ thống bắt được lỗi và trả về HTTP 500, không bị sập.	Pass*	Cần cải thiện thông báo lỗi cho người dùng.
TC-F-04	Tên file có ký tự đặc biệt được xử lý chính xác.	Pass	
TC-F-05	Hệ thống không có cơ chế giới hạn dung lượng file tải lên.	Fail	Bug #F-1
TC-F-06	Thiếu xác thực MIME type, chỉ kiểm tra đuôi file.	Fail	Bug #F-1
TC-F-07	Endpoint upload không được bảo vệ bằng API Key.	Fail	Bug #SEC-1
TC-C-01	Luồng RAG cơ bản hoạt động, truy xuất được thông tin từ tài liệu.	Pass	
TC-C-02	Chatbot trả lời đúng khi không tìm thấy thông tin.	Pass	
TC-C-03	Nút gửi được vô hiệu hóa ở Frontend.	Pass	Kiểm thử tự động với Playwright đã xác nhận.
TC-C-04	API query chỉ nhận question, không nạp lịch sử hội thoại.	Fail	Bug #RAG-1
TC-C-05	API chat là request-response đồng bộ, không thể hủy ngang.	Fail	Bug #RAG-2
TC-C-06	Prompt chưa đủ tốt để hướng dẫn LLM tổng hợp thông tin.	Fail	Bug #RAG-3
TC-C-07	Backend không có validation cho truy vấn rỗng, có thể gây lãng phí tài nguyên.	Fail	Bug #F-1
TC-C-08	Endpoint RAG không nhận conversation_id , nên không thể kiểm tra.	Fail	(Là hệ quả của Bug #RAG-1)
TC-M-01	API POST /conversation/create hoạt động đúng.	Pass	
TC-M-02	API GET /{conv_id}/history hoạt động đúng.	Pass	
TC-M-03	API DELETE /{conv_id} hoạt động đúng.	Pass	
TC-M-04	API POST /conversation/re-name xử lý thành công.	Pass	
TC-M-05	Endpoint DELETE xử lý đúng trường hợp không tìm thấy, trả về 404.	Pass	
TC-M-06	Endpoint trả về mảng rỗng như mong đợi.	Pass	

Table 6.7: Kết quả thực thi các ca kiểm thử

6.3.2 Phân tích lỗi và Đề xuất cải tiến

Qua quá trình kiểm thử, một số vấn đề cốt lõi đã được xác định, ảnh hưởng đến chức năng, độ ổn định và bảo mật của hệ thống.

- **Bug #RAG-1: Chatbot thiếu khả năng ghi nhớ ngữ cảnh (Conversational Memory).**
 - **Mô tả:** Hệ thống xử lý mỗi câu hỏi một cách độc lập, không xem xét các lượt trao đổi trước đó trong cùng một hội thoại. Điều này làm giảm trải nghiệm người dùng, không cho phép các câu hỏi nối tiếp có tham chiếu.
 - **Nguyên nhân:** Endpoint `/rag/query` và service `RAGService.query` chỉ nhận `question` làm đầu vào. Không có logic nào được triển khai để tải lịch sử trò chuyện và đưa vào ngữ cảnh cho LLM.
 - **Đề xuất khắc phục:** Sửa đổi API `/rag/query` để nhận thêm `conversation_id`. Trước khi gọi LLM, tải k cặp hỏiđáp gần nhất từ hội thoại đó và định dạng lại chúng như một phần của prompt ngữ cảnh.
- **Bug #RAG-2: Chức năng "Dừng tạo câu trả lời" không hoạt động ở Backend.**
 - **Mô tả:** Giao diện có nút "Dừng", nhưng nó không thể thực sự ngắt quá trình sinh câu trả lời đang diễn ra ở máy chủ.
 - **Nguyên nhân:** API chat sử dụng giao thức HTTP request-response đồng bộ. Một khi yêu cầu đã được gửi đi, client phải chờ cho đến khi toàn bộ quá trình xử lý hoàn tất.
 - **Đề xuất khắc phục:** Tái cấu trúc API chat để sử dụng **ServerSent Events (SSE)**. SSE tạo một kết nối bền vững cho phép server gửi dữ liệu (từng token) về client theo thời gian thực và cho phép client hủy kết nối bất kỳ lúc nào.
- **Bug #RAG-3: Khả năng suy luận và tổng hợp thông tin của Chatbot còn hạn chế.**
 - **Mô tả:** Chatbot gặp khó khăn với các câu hỏi yêu cầu tổng hợp dữ liệu từ nhiều đoạn văn bản khác nhau.
 - **Nguyên nhân:** Prompt hiện tại chỉ đơn thuần ghép nối các chunks văn bản tìm được. Nó thiếu các chỉ dẫn rõ ràng để LLM thực hiện các tác vụ suy luận phức tạp.
 - **Đề xuất khắc phục:** Áp dụng kỹ thuật Prompt Engineering. Cải tiến mẫu prompt để chỉ dẫn tường minh cho LLM, ví dụ: "Dựa vào các đoạn thông tin sau, hãy phân tích, tổng hợp và trả lời câu hỏi của người dùng một cách ngắn gọn..."
- **Bug #F-1: Thiếu cơ chế xác thực đầu vào (Input Validation) cho các tệp tải lên.**
 - **Mô tả:** Endpoint tải file không kiểm tra định dạng, MIME type, và kích thước của file. Điều này có thể gây lỗi ở các bước xử lý sau, lãng phí tài nguyên và tạo ra lỗ hổng bảo mật.

- **Nguyên nhân:** Endpoint [POST/files/upload](#) trong [api/file/routes.py](#) xử lý file ngay mà không qua các bước kiểm tra cần thiết.
- **Đề xuất khắc phục:** Bổ sung logic validation:
 1. Kiểm tra phần mở rộng file và [file.content_type](#) với một danh sách các loại MIME được cho phép (ví dụ: [application/pdf](#)).
 2. Đọc file theo từng khối (chunk) để kiểm tra kích thước thực tế, tránh tải toàn bộ file lớn vào bộ nhớ, và so sánh với một ngưỡng đã định cấu hình.
 3. Trả về lỗi HTTP 400 Bad Request nếu không hợp lệ.
- **Bug #SEC-1: Lỗ hổng bảo mật do thiếu xác thực API.**
 - **Mô tả:** Các endpoint quan trọng, đặc biệt là [POST/files/upload](#), không yêu cầu xác thực API key.
 - **Nguyên nhân:** Thiếu một lớp dependency injection trong FastAPI để kiểm tra header [x-api-key](#) trên các route cần bảo vệ.
 - **Đề xuất khắc phục:** Tạo một hàm dependency chung để xác thực API key và áp dụng nó vào các router hoặc các endpoint cụ thể cần bảo vệ.

6.4 Tổng kết và Hướng phát triển tương lai

6.4.1 Tổng kết

Kết quả kiểm thử thực tế dựa trên phân tích mã nguồn cho thấy một bức tranh rõ ràng: sản phẩm có một nền tảng chức năng cơ bản ổn định, nhưng tồn tại những lỗ hổng đáng kể về trí thông minh, khả năng tương tác và độ tin cậy. Với **8/13 (61.5%)** ca kiểm thử thành công, hệ thống đã bộc lộ **5 lỗi quan trọng** cần được ưu tiên khắc phục.

6.4.2 Hướng phát triển tương lai

Để nâng cao chất lượng và sự hoàn thiện của sản phẩm, các hoạt động sau được đề xuất cho giai đoạn phát triển tiếp theo:

- **Khắc phục các lỗi đã phát hiện:** Ưu tiên xử lý toàn bộ 5 bug đã được ghi nhận theo thứ tự nghiêm trọng:
 1. **Bug #RAG-1, #RAG-2, #RAG-3:** Cải thiện trí thông minh và khả năng tương tác cốt lõi (bộ nhớ, dừng tác vụ, tổng hợp thông tin).
 2. **Bug #F-1:** Gia cố hệ thống bằng cách thêm các lớp xác thực đầu vào cho chức năng tải file.
- **Xây dựng bộ kiểm thử tự động (Automation Testing):**
 - Sử dụng [Pytest](#) để viết unit test và integration test cho các API của Backend, đặc biệt cho các logic đã sửa lỗi để chống tái phát (regression).

- Sử dụng một framework như [Cypress](#) hoặc [Playwright](#) để tự động hóa các kịch bản kiểm thử E2E cho Frontend.
- **Kiểm thử hiệu năng (Performance Testing):** Sau khi hệ thống ổn định, sử dụng các công cụ như [k6](#) hoặc [JMeter](#) để thực hiện kiểm thử tải, đánh giá sức chịu tải của hệ thống và thời gian phản hồi dưới áp lực.

CHƯƠNG 7 TRIỂN KHAI VÀ CÀI ĐẶT

7.1 Tổng quan

Nội dung chương này trình bày chi tiết về quy trình cài đặt môi trường phát triển cục bộ và phương pháp triển khai hệ thống hoàn chỉnh lên các môi trường khác nhau. Hệ thống được thiết kế để có thể vận hành trên máy tính cá nhân của lập trình viên hoặc được đóng gói và phân phối một cách nhất quán thông qua công nghệ container hóa với Docker.

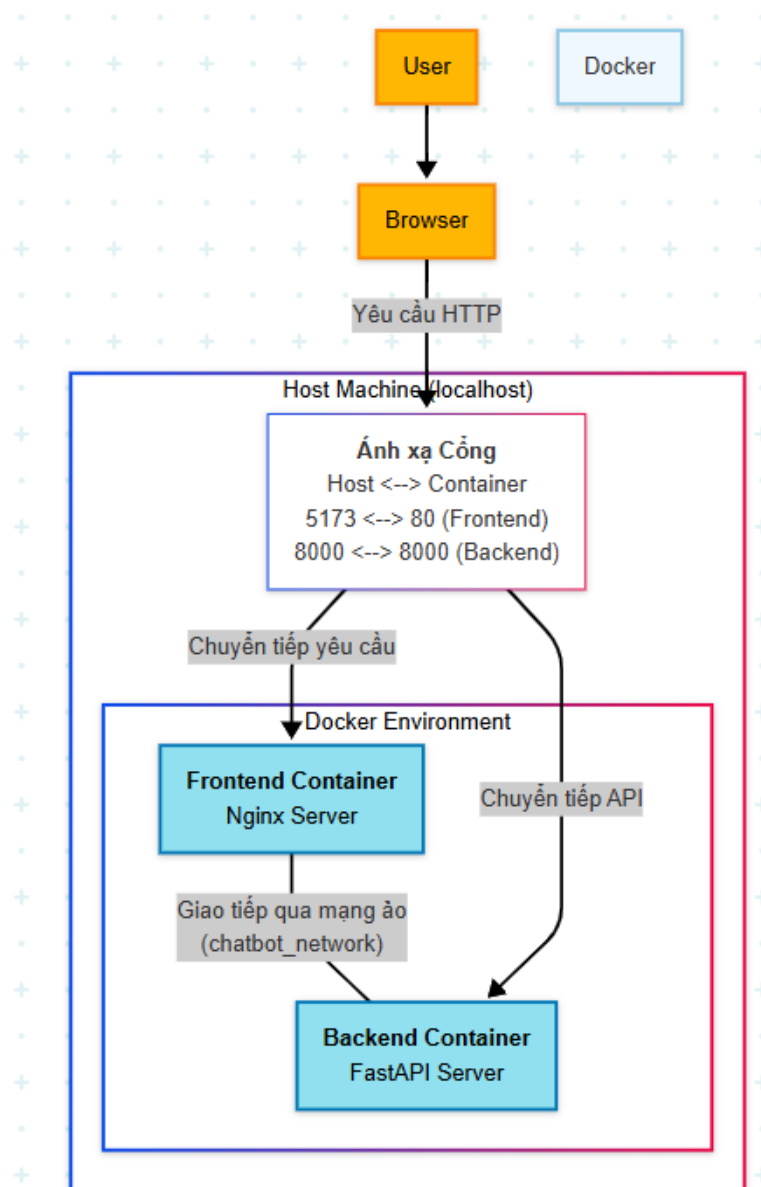


Figure 7.1: Sơ đồ kiến trúc triển khai hệ thống bằng Docker Compose.

7.2 Môi trường Phát triển Cục bộ (Local Development)

Đây là môi trường dành cho các lập trình viên trực tiếp phát triển và gỡ lỗi các tính năng của hệ thống. Để thiết lập môi trường này, cần đảm bảo các công cụ và nền tảng sau đã được cài đặt.

7.2.1 Công cụ yêu cầu

- [Node.js](#) (phiên bản 18.x trở lên): Môi trường thực thi JavaScript, cần thiết để chạy giao diện người dùng (Frontend) được xây dựng bằng Vite và React.
- [Python](#) (phiên bản 3.10 trở lên): Ngôn ngữ lập trình chính cho phần xử lý logic phía máy chủ (Backend).
- [pip](#): Trình quản lý gói mặc định của Python, được sử dụng để cài đặt các thư viện phụ thuộc cho Backend.
- [Git](#): Hệ thống quản lý phiên bản phân tán, dùng để tải mã nguồn của dự án từ kho chứa trực tuyến.

7.2.2 Hướng dẫn cài đặt

1. **Tải mã nguồn:** Sử dụng Git để sao chép (clone) toàn bộ mã nguồn của dự án về máy tính cá nhân từ kho chứa tại <https://github.com/TranAnh35/MinBot>.
2. **Cài đặt Backend:** Trong thư mục [src/backend](#), cần tạo một môi trường ảo Python để cô lập các thư viện. Sau đó, kích hoạt môi trường ảo và cài đặt tất cả các gói phụ thuộc được định nghĩa trong file [requirements.txt](#).
Lưu ý: Việc sử dụng môi trường ảo (virtual environment) là một thông lệ tốt, giúp tránh xung đột phiên bản giữa các thư viện của nhiều dự án khác nhau trên cùng một máy tính.
3. **Cài đặt Frontend:** Di chuyển vào thư mục [src/frontend](#) và thực hiện cài đặt các gói phụ thuộc của Node.js bằng trình quản lý gói [npm](#).
4. **Thiết lập biến môi trường:** Các thông tin nhạy cảm như API key không được lưu trực tiếp trong mã nguồn. Cần tạo một file [.env](#) trong thư mục [src/backend](#) và điền các giá trị cần thiết. File này thường được tạo bằng cách sao chép từ một file mẫu là [.env.example](#).
5. **Khởi chạy hệ thống:** Hệ thống bao gồm hai tiến trình độc lập: server backend và server frontend, cần được khởi chạy trên hai cửa sổ dòng lệnh (terminal) riêng biệt.
 - **Chạy Backend:** Server backend được phục vụ bởi [uvicorn](#), một máy chủ ASGI (Asynchronous Server Gateway Interface) hiệu năng cao. Sau khi khởi chạy, API sẽ có thể được truy cập tại <http://127.0.0.1:8000>.

- **Chạy Frontend:** Server frontend được khởi chạy bằng [Vite](#), một công cụ xây dựng hiện đại cho các ứng dụng web. Giao diện ứng dụng sẽ có thể được truy cập tại <http://localhost:5173>.

7.3 Triển khai với Docker

Để đảm bảo tính nhất quán, di động và dễ dàng triển khai trên mọi môi trường từ Staging đến Production, dự án sử dụng công nghệ container hóa với Docker. Giải pháp này đóng gói ứng dụng cùng tất cả các phụ thuộc của nó vào một "container" có thể chạy ở bất cứ đâu.

Người dùng tương tác với trình duyệt, gửi yêu cầu đến cổng 5173 của máy chủ. Yêu cầu này được chuyển đến container [frontend](#) (chạy Nginx). Giao diện người dùng sau đó sẽ gọi API đến cổng 8000, yêu cầu này đi qua mạng ảo [chatbot_network](#) và được chuyển đến container [backend](#) (chạy FastAPI) để xử lý.

7.3.1 Phân tích [Dockerfile](#)

Dự án sử dụng hai [Dockerfile](#) riêng biệt để xây dựng image cho frontend và backend, đều áp dụng kỹ thuật *multi-stage build* để tối ưu hóa kích thước và tăng cường bảo mật.

- [Dockerfile.backend](#): Image của Backend được xây dựng dựa trên [python:3.10-slim](#). Giai đoạn đầu ('builder') thực hiện cài đặt các công cụ biên dịch cần thiết và toàn bộ thư viện Python từ file [requirements.txt](#) vào một môi trường tạm. Giai đoạn cuối cùng chỉ sao chép mã nguồn của ứng dụng và các thư viện đã được cài đặt từ giai đoạn trước vào một image [slim](#) tối giản. Cách tiếp cận này giúp giảm đáng kể kích thước image cuối cùng, loại bỏ các công cụ không cần thiết cho môi trường production, qua đó tăng cường tính bảo mật.
- [Dockerfile.frontend](#): Tương tự, image Frontend cũng được xây dựng qua hai giai đoạn. Giai đoạn đầu ('build') sử dụng image [node:18-alpine](#) để biên dịch mã nguồn React/Vite thành các tệp HTML, CSS, và JavaScript tĩnh. Giai đoạn thứ hai sử dụng một image [Nginx:stable-alpine](#) siêu nhẹ, chỉ làm nhiệm vụ phục vụ các tệp tĩnh này. Kết quả là một image production cực kỳ nhỏ gọn và an toàn, chỉ chứa những gì cần thiết để chạy ứng dụng.

7.3.2 Phân tích [docker-compose.yml](#)

File [docker-compose.yml](#) dùng để định nghĩa và điều phối các container (dịch vụ) của ứng dụng, bao gồm:

- **Dịch vụ backend:** Được xây dựng từ [Dockerfile.backend](#), ánh xạ cổng 8000 của container ra máy chủ. File [.env](#) chứa các biến môi trường nhạy cảm được

nạp vào container, và mã nguồn được liên kết thông qua [volume](#) để hỗ trợ "live-reload" trong môi trường phát triển.

- **Dịch vụ frontend:** Được xây dựng từ [Dockerfile.frontend](#), ánh xạ cổng 80 (của Nginx) ra cổng 5173 trên máy chủ. Dịch vụ này được cấu hình để khởi động sau ([depends_on](#)) dịch vụ [backend](#).
- **Mạng (networks):** Cả hai dịch vụ được kết nối qua một mạng ảo riêng tên là [chatbot_network](#) (sử dụng driver [bridge](#)), cho phép chúng giao tiếp với nhau một cách an toàn và tách biệt khỏi các container khác trên cùng máy chủ.

7.3.3 Hướng dẫn triển khai bằng Docker

1. **Cài đặt Docker:** Tải và cài đặt [DockerDesktop](#) từ trang chủ của Docker.
2. **Build và khởi chạy container:** Thực thi lệnh [docker-composeup--build-d](#) tại thư mục gốc của dự án. Lệnh này sẽ tự động xây dựng các image (nếu chưa có), sau đó khởi chạy các container ở chế độ nền.

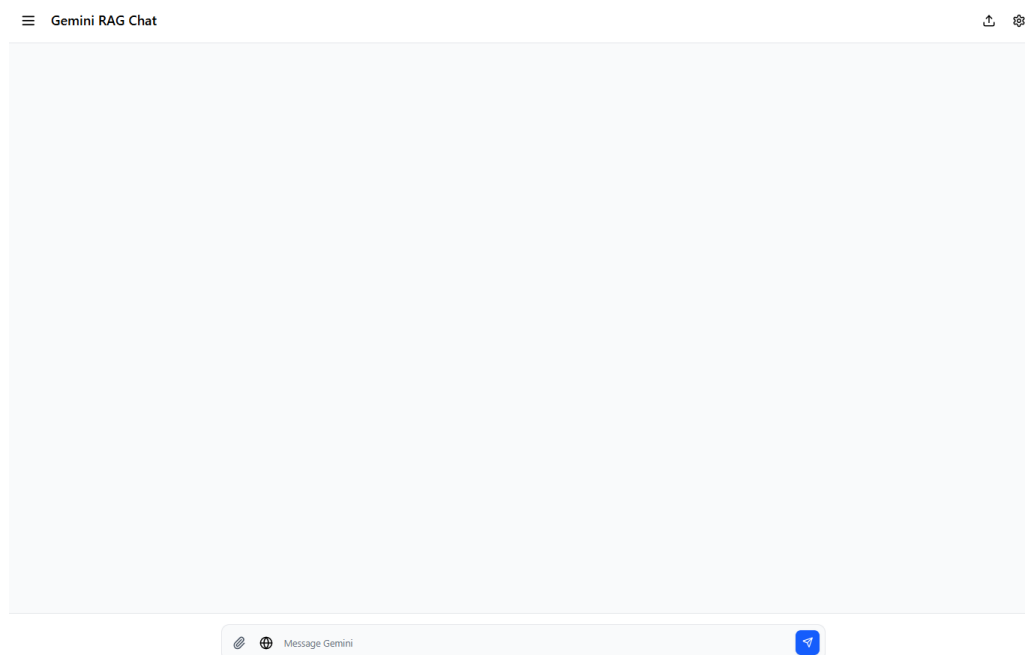


Figure 7.2: Giao diện chính của ứng dụng MinBot sau khi triển khai thành công.

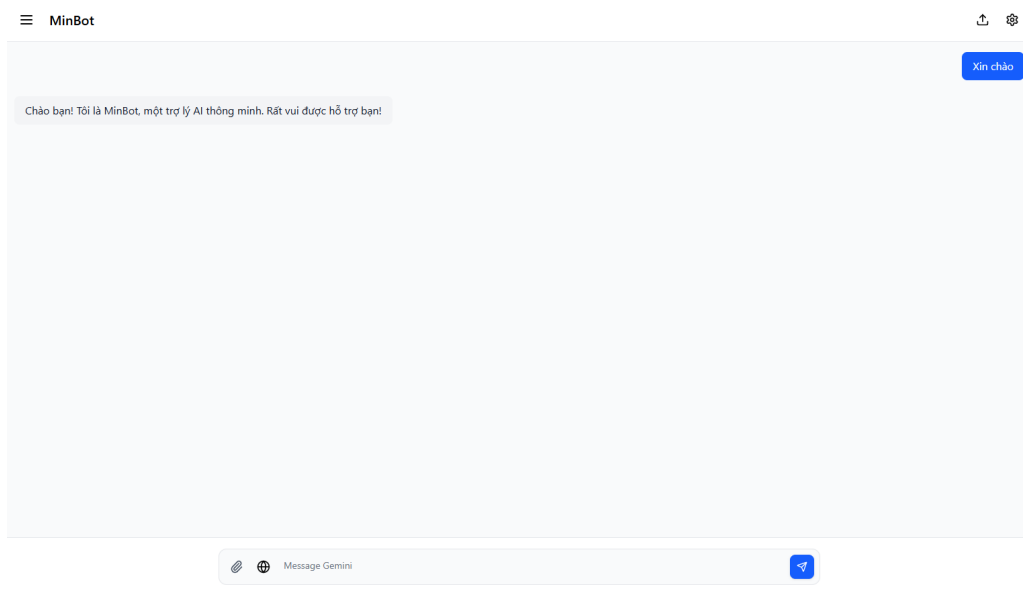


Figure 7.3: Giao diện chat của MinBot.

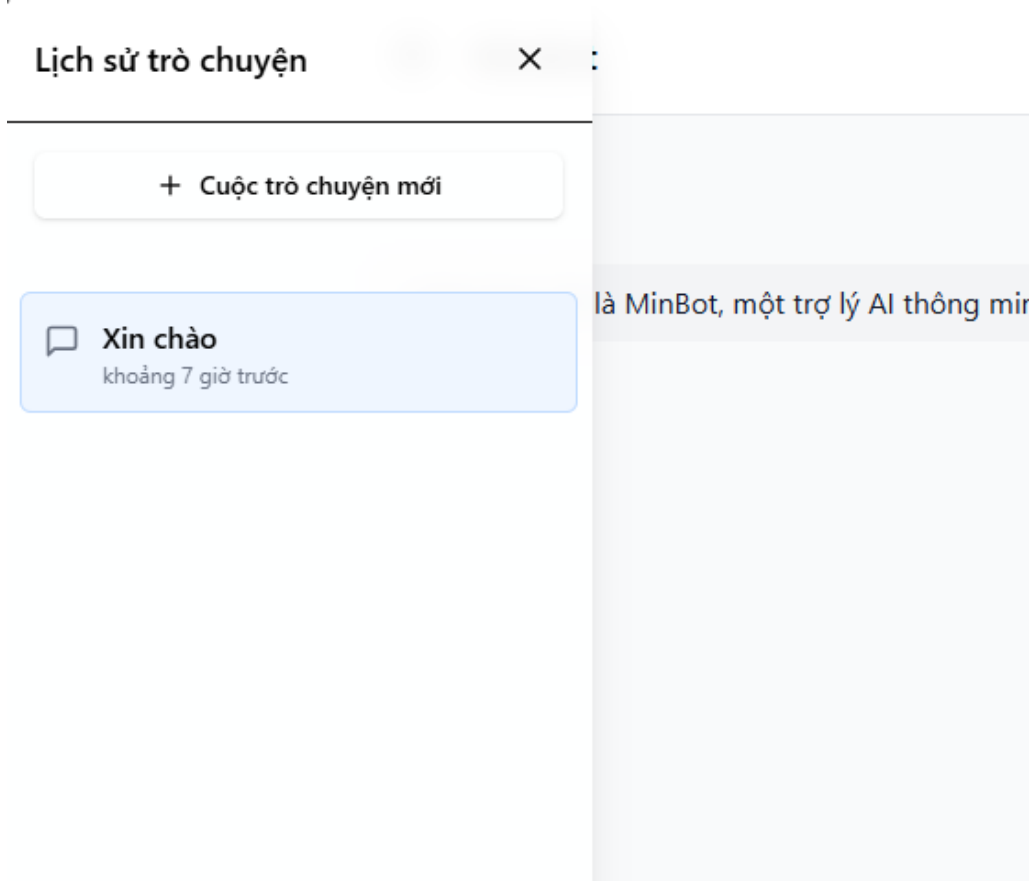


Figure 7.4: Giao diện conversation của MinBot.

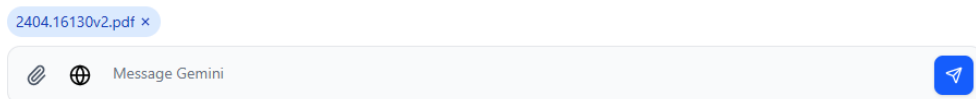


Figure 7.5: Giao diện tài liệu được gắn vào message của MinBot.

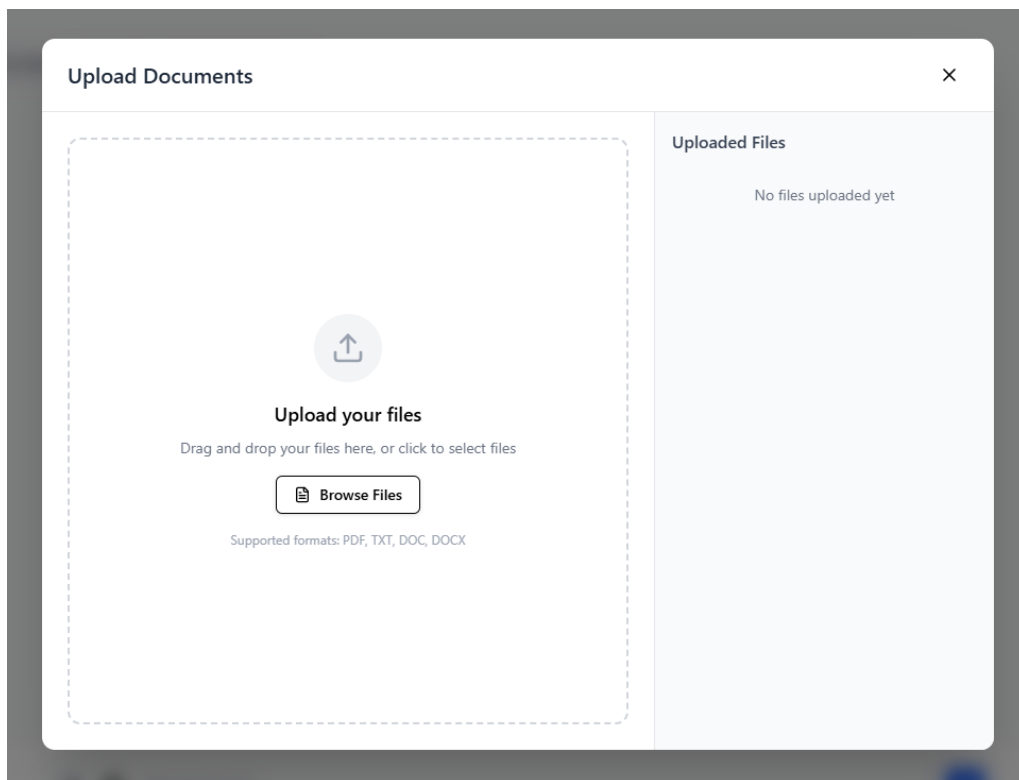


Figure 7.6: Giao diện upload tài liệu của MinBot.

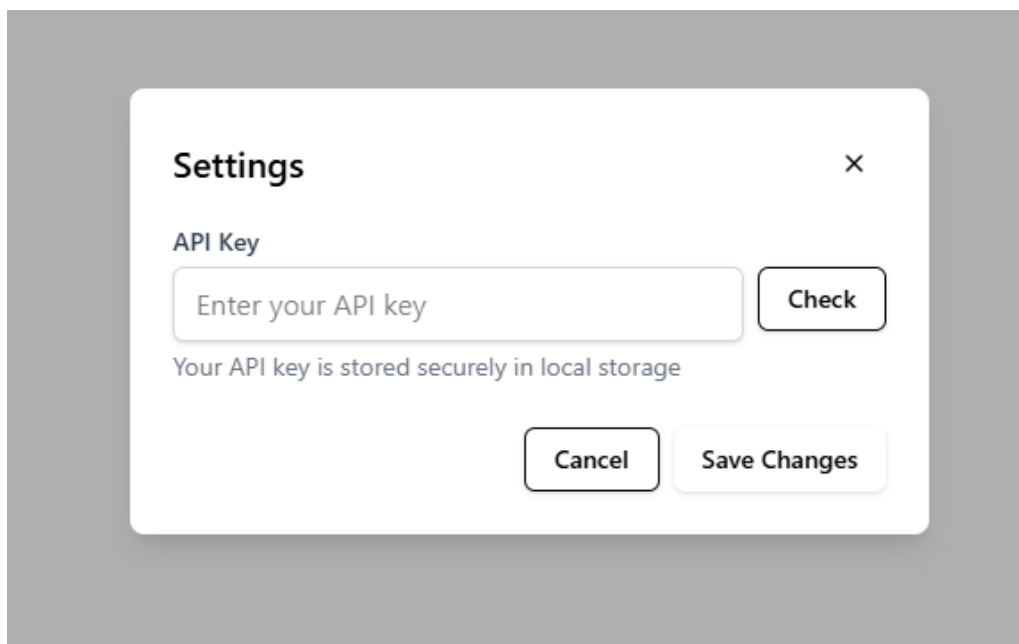


Figure 7.7: Giao diện cài đặt của MinBot.

Agent System 1.0.0 OAS 3.1
/openapi.json

RAG	▼
Files	▼
WebSearch	▼
Generative Content	▼
API Key	▼
Conversations	▼
System Management	▼
default	▼

Figure 7.8: Giao diện tài liệu API (Swagger UI) của backend tại địa chỉ [/docs](#).

CHƯƠNG 8 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Sau một thời gian nghiên cứu và thực hiện, dự án **”Xây dựng Ứng dụng Chatbot thông minh”** đã hoàn thành các mục tiêu chính đã đề ra, xây dựng thành công một sản phẩm khả thi tối thiểu (MVP) với nền tảng công nghệ vững chắc. Chương cuối cùng này sẽ tổng kết lại các thành tựu đã đạt được, nhìn nhận thẳng thắn những hạn chế còn tồn tại dựa trên kết quả kiểm thử, và đề xuất một lộ trình phát triển rõ ràng trong tương lai.

8.1 Kết quả Đạt được

Dự án đã thành công trong việc kiến tạo một hệ thống Chatbot RAG hoàn chỉnh, từ backend đến frontend, có khả năng triển khai và hoạt động độc lập. Đây là một nền tảng vững chắc cho các phát triển sâu hơn.

1. Kiến trúc Backend vững chắc và có khả năng mở rộng:

- Xây dựng hệ thống API bằng [FastAPI](#) theo kiến trúc module hóa, phân tách rõ ràng các tầng nghiệp vụ (services), giao tiếp (API routes) và mô hình dữ liệu (models). Điều này giúp hệ thống dễ dàng bảo trì, kiểm thử và tích hợp thêm các tính năng mới trong tương lai.
- Tận dụng khả năng xử lý bất đồng bộ của [FastAPI](#) để tối ưu hóa các tác vụ I/O, đảm bảo hiệu năng và khả năng đáp ứng của hệ thống.

2. Giao diện người dùng hiện đại và đáp ứng:

- Phát triển một giao diện người dùng sạch, trực quan bằng [React](#) và [TypeScript](#), cung cấp trải nghiệm tương tác mượt mà cho các chức năng cốt lõi: tải tài liệu, trò chuyện và quản lý các phiên hội thoại.
- Việc áp dụng [TypeScript](#) và kiến trúc Component-based giúp đảm bảo sự chặt chẽ của mã nguồn, giảm thiểu lỗi và tăng tốc độ phát triển.

3. Hiện thực hóa thành công luồng xử lý RAG hoàn chỉnh:

- Triển khai thành công luồng xử lý [RAG](#) end-to-end: từ việc nạp, phân đoạn ([chunking](#)), và vector hóa tài liệu bằng các mô hình [embedding](#), đến việc lưu trữ và truy xuất hiệu quả bằng cơ sở dữ liệu vector [FAISS](#).
- Hệ thống chứng tỏ được khả năng ”trả lời dựa trên ngữ cảnh”, giảm thiểu hiện tượng ảo giác (hallucination) của LLM bằng cách cung cấp các thông tin liên quan trích xuất từ tài liệu.

4. Đảm bảo tính di động và khả năng tái lập bằng Docker:

- Toàn bộ ứng dụng đã được đóng gói hoàn chỉnh bằng [Docker](#) và [Docker Compose](#), thống nhất môi trường phát triển và triển khai.
- Cách tiếp cận này giúp đơn giản hóa tối đa quá trình cài đặt, cho phép bất kỳ ai cũng có thể khởi chạy toàn bộ hệ thống chỉ với một lệnh, đảm bảo tính nhất quán và loại bỏ các xung đột về môi trường.

8.2 Hạn chế của dự án

Bên cạnh những kết quả tích cực, quá trình kiểm thử ở chương trước đã chỉ ra những điểm yếu cốt lõi của hệ thống. Việc nhìn nhận thẳng thắn các hạn chế này là bước đầu tiên để hoàn thiện sản phẩm.

- **Hạn chế về Trí tuệ và Tương tác (Core Intelligence & Interaction):** Đây là nhóm hạn chế nghiêm trọng nhất, ảnh hưởng trực tiếp đến trải nghiệm và sự "thông minh" của chatbot.
 - **Thiếu bộ nhớ hội thoại (Bug#RAG-1):** Hệ thống xử lý mỗi câu hỏi một cách độc lập, không có khả năng ghi nhớ và sử dụng ngữ cảnh từ các trao đổi trước đó.
 - **Không thể hủy tác vụ (Bug#RAG-2):** Do sử dụng kiến trúc request-response đồng bộ, người dùng không thể dừng việc tạo câu trả lời đang diễn ra ở backend, gây ra trải nghiệm chờ đợi không cần thiết.
 - **Khả năng suy luận còn yếu (Bug#RAG-3):** Prompt hiện tại chưa được tối ưu để hướng dẫn LLM tổng hợp thông tin từ nhiều nguồn dữ liệu khác nhau, dẫn đến hạn chế trong việc trả lời các câu hỏi phức tạp.
- **Hạn chế về Độ tin cậy và An toàn (Reliability & Security):** Hệ thống còn thiếu các cơ chế xác thực và bảo vệ cơ bản, tiềm ẩn rủi ro về ổn định và bảo mật.
 - **Thiếu xác thực đầu vào cho file (Bug#F-1):** Hệ thống không kiểm tra định dạng, loại MIME, hay giới hạn dung lượng file tải lên. Điều này có thể bị lạm dụng để làm cạn kiệt tài nguyên máy chủ hoặc gây lỗi nghiêm trọng ở tầng xử lý dữ liệu.
 - **Lỗ hổng bảo mật API (Bug#SEC-1):** Endpoint upload file không được bảo vệ, cho phép bất kỳ ai cũng có thể tải file lên hệ thống mà không cần xác thực.
- **Hạn chế về Chức năng và Khả năng mở rộng:**
 - **Chưa có hệ thống người dùng:** Thiếu cơ chế đăng ký/dăng nhập, khiến hệ thống chỉ có thể phục vụ một người dùng tại một thời điểm và không thể cá nhân hóa trải nghiệm.
 - **Chất lượng RAG có thể tối ưu hơn:** Các chiến lược phân đoạn văn bản và truy xuất thông tin hiện tại còn cơ bản, chưa được tối ưu cho các loại tài liệu đa dạng.

8.3 Lộ trình Phát triển trong Tương lai

Từ những hạn chế đã phân tích, một lộ trình phát triển được đề xuất bao gồm hai giai đoạn chính, nhằm biến sản phẩm từ một MVP thành một ứng dụng hoàn thiện và mạnh mẽ.

8.3.1 Giai đoạn 1: Hoàn thiện Sản phẩm Cốt lõi

Đây là giai đoạn ưu tiên hàng đầu, tập trung vào việc khắc phục các lỗi nghiêm trọng đã được xác định để nâng cao chất lượng và độ tin cậy của sản phẩm.

1. **Khắc phục Bug#RAG-1 - Tích hợp ConversationMemory:** Sửa đổi API để nạp lịch sử chat gần nhất vào prompt, cung cấp ngữ cảnh đầy đủ cho LLM.
2. **Khắc phục Bug#RAG-2 - Triển khai Streaming và Hủy tác vụ:** Chuyển đổi API chat từ HTTP request sang **Server-Sent Events (SSE)** để cho phép trả lời theo dòng (streaming) và nhận tín hiệu hủy từ client.
3. **Khắc phục Bug#F-1 & Bug#SEC-1 - Gia cố API Upload:** Bổ sung logic xác thực toàn diện (định dạng, kích thước, MIME type) và triển khai lớp bảo vệ bằng API Key cho endpoint tải file.
4. **Khắc phục Bug#RAG-3 - Cải tiến Prompt Engineering:** Thiết kế lại các mẫu prompt theo những kỹ thuật tiên tiến như **Chain-of-Thought**, nhằm hướng dẫn LLM thực hiện các tác vụ suy luận và tổng hợp thông tin từ nhiều nguồn một cách tường minh và hiệu quả hơn.

8.3.2 Giai đoạn 2: Nâng cấp và Mở rộng

Sau khi nền tảng đã ổn định, các tính năng nâng cao sẽ được phát triển để mở rộng khả năng của hệ thống.

- **Nâng cao chất lượng RAG:**
 - **Hybrid Search:** Kết hợp tìm kiếm vector (semantic) và tìm kiếm từ khóa (keyword) để tăng độ chính xác truy xuất.
 - **Re-ranking Model:** Sử dụng một mô hình sắp xếp lại (re-ranker) để tối ưu hóa thứ tự của các tài liệu được truy xuất trước khi đưa vào LLM.
- **Nâng cấp Trải nghiệm người dùng (UI/UX):**
 - **Source Highlighting:** Hiển thị và tô sáng nguồn thông tin trong tài liệu gốc, tăng tính minh bạch cho câu trả lời.
 - **User Feedback:** Thêm cơ chế cho phép người dùng đánh giá câu trả lời, tạo ra dữ liệu quý giá cho việc tinh chỉnh mô hình sau này (RLHF).
- **Xây dựng hệ thống Đa người dùng (Multi-tenancy):** Tích hợp cơ chế xác thực **JWT**, quản lý phiên làm việc và phân quyền để mỗi người dùng có không gian làm việc và cơ sở tri thức riêng.

- **Tự động hóa Quy trình phát triển (CI/CD):** Thiết lập quy trình Tích hợp và Triển khai liên tục bằng [GitHubActions](#) để tự động hóa việc kiểm thử và đóng gói ứng dụng, đảm bảo quy trình phát triển chuyên nghiệp và hiệu quả.
- **Tối ưu hóa Chi phí và Hiệu năng:** Nghiên cứu và tích hợp các mô hình embedding hoặc LLM nhỏ hơn nhưng vẫn hiệu quả (có thể thông qua *fine-tuning*) nhằm mục đích giảm độ trễ của câu trả lời và tối ưu chi phí vận hành hệ thống.

Tóm lại, dự án đã đặt một viên gạch nền móng vững chắc. Với lộ trình phát triển rõ ràng, dự án hoàn toàn có tiềm năng để trở thành một giải pháp Chatbot mạnh mẽ, thông minh và đáng tin cậy.

Tài liệu tham khảo

- [1] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 \mkbibbrackets{cs.CL}. URL: <https://arxiv.org/abs/1706.03762>.
- [2] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 \mkbibbrackets{cs.CL}. URL: <https://arxiv.org/abs/1810.04805>.
- [3] Alec Radford and Karthik Narasimhan. “Improving Language Understanding by Generative Pre-Training”. In: (2018).
- [4] Jared Kaplan et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 \mkbibbrackets{cs.LG}. URL: <https://arxiv.org/abs/2001.08361>.
- [5] Long Ouyang et al. *Training language models to follow instructions with human feedback*. 2022. arXiv: 2203.02155 \mkbibbrackets{cs.CL}. URL: <https://arxiv.org/abs/2203.02155>.
- [6] OpenAI. “OpenAI o3 and o4-mini System Card”. In: (2025). URL: <https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf>.
- [7] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 \mkbibbrackets{cs.CL}. URL: <https://arxiv.org/abs/2005.11401>.
- [8] Matthijs Douze et al. *The Faiss library*. 2025. arXiv: 2401.08281 \mkbibbrackets{cs.LG}. URL: <https://arxiv.org/abs/2401.08281>.